Network Intrusion Detection with Quantum Kernel Support Vector Machines

Project Documentation and Improvement Report

CIC-DDoS2019 • ZZ Entangling Feature Map • Google Cirq

This report explains the whole project in detail: the dataset, the classical and quantum machine-learning theory behind it, and the original code and its results. It then documents every improvement made to the code and why, with derivations and verification, and reports a complete re-evaluation under a leakage-free, statistically controlled protocol. Exact commands to reproduce every result are given in `TESTING.md` and Appendix C.

Department of Computer Engineering

October 28, 2026

Abstract

The project classifies network flows from the CIC-DDoS2019 benchmark as benign or as one of five DDoS attack types. It uses a quantum support vector machine (QSVM) whose kernel is the fidelity between quantum states prepared by a ZZ entangling feature map, simulated with Google Cirq. The original implementation reported that the entangled kernel beats a classical RBF-SVM by +1.8 points (binary) and +12 points (6-class). We audited the code and data and found five issues that affect those conclusions: (i) 65% of the rows in the data file are exact duplicates, so about 30% of test flows have an identical copy in the training set; (ii) a min–max scaler was fitted on the whole dataset before the train/test split; (iii) heavy-tailed flow features were mapped linearly onto rotation angles, which wastes most of the angle range; (iv) neither the quantum kernel's bandwidth nor the SVM hyperparameters were tuned, and the classical baseline was never tuned at all; (v) each kernel entry simulated two full circuits, so a 1600×1600 kernel at 16 qubits cost an estimated 44 hours.

We kept the project's architecture and fixed each issue: deduplication, train-only preprocessing with a signed log transform, a bandwidth parameter α tuned by cross-validation together with C (and C, γ for the RBF baseline), and an exact NumPy statevector engine that uses the diagonal structure of the ZZ map. The engine matches Cirq to 10^{-7}, simulates each sample once instead of twice per kernel entry, and computes the same 16-qubit kernels in about 26 seconds. We also added macro-F1, MCC, false-alarm and detection rates, multi-seed confidence intervals, McNemar significance tests, kernel-concentration and kernel–target-alignment diagnostics, the geometric-difference test of Huang et al., a Random-Forest reference, unit tests and a resumable benchmark suite.

Under the corrected protocol, all models reach high accuracy, and the quantum kernel performs on par with a tuned classical kernel on the same features. The large advantages reported earlier come mainly from the leakage and from the untuned baseline, not from entanglement. The diagnostics explain why, and they show where a quantum kernel could still help.

Contents

1 Introduction **4**
1.1 Motivation . . . 4
1.2 The project . . . 4
1.3 Goal of this report . . . 4

2 Background **5**
2.1 DDoS attacks in the dataset . . . 5
2.2 Flow-based detection and CICFlowMeter . . . 6
2.3 Support vector machines and kernels . . . 6
2.4 Quantum computing essentials . . . 6
2.5 Quantum feature maps and kernels . . . 7
2.6 Kernel concentration and bandwidth . . . 8
2.7 When can a quantum kernel help? . . . 8

3 The Dataset in This Repository **8**
3.1 Where the data lives . . . 8
3.2 Class composition and duplicates . . . 9
3.3 Features . . . 9
3.4 Tasks . . . 10

4 The Original Pipeline and Its Audit **10**
4.1 Architecture . . . 10
4.2 Reported results . . . 11
4.3 Audit findings . . . 12

5 Improvements I: Data Integrity **13**
5.1 Duplicate removal (I1) . . . 14
5.2 Leakage-free preprocessing (I5) . . . 14
5.3 Signed log transform (I6) . . . 14
5.4 Reproducible sampling and real sample sizes (I8, I12, I13) . . . 15

6 Improvements II: Computation **15**
6.1 Simulate each sample once (statevector caching) . . . 15
6.2 An exact fast engine for the ZZ feature map . . . 16
6.3 Other kernels on the same engine . . . 17
6.4 Verification . . . 17
6.5 Speed . . . 17

7 Improvements III: Modelling **18**
7.1 Kernel bandwidth α (I3) . . . 18
7.2 Cross-validated tuning with equal budgets (I2) . . . 19

7.3 Entanglement topology and depth . . . 19
7.4 All three paper methods in one pipeline (I10) . . . 19
7.5 Stronger classical references . . . 19

8 Improvements IV: Evaluation and Software **19**
8.1 Metrics (I11) . . . 19
8.2 Repetition and significance (I7) . . . 20
8.3 Kernel diagnostics . . . 20
8.4 Software engineering . . . 21

9 Results **22**
9.1 Headline: the same circuits under the original and the improved protocol . . . 22
9.2 Ablation: which fix changes what . . . 24
9.3 All metrics for the improved ZZ pipeline . . . 25
9.4 Bandwidth α . . . 25
9.5 Number of qubits . . . 27
9.6 Shot noise . . . 28
9.7 Circuit architecture and feature set . . . 29
9.8 Learning curves . . . 30
9.9 Kernel diagnostics . . . 30

10 Discussion **31**
10.1 Does the quantum kernel help on this task? . . . 31
10.2 What went wrong in the original evaluation . . . 32
10.3 Threats to validity . . . 32
10.4 Future work . . . 33
10.5 Conclusion . . . 33

A Command-line reference **34**

B File-by-file reference **34**

C Reproduction commands **35**

D Glossary **35**

1 Introduction

1.1 Motivation

Distributed Denial-of-Service (DDoS) attacks overwhelm a target with traffic from many sources. They remain among the most frequent and most costly attacks on networked infrastructure. Modern DDoS campaigns often use *reflection and amplification*: the attacker sends small requests with a spoofed source address to public servers (LDAP, MSSQL, Portmap/rpcbind, and so on), and those servers send much larger responses to the victim. Signature-based detection struggles against these attacks because the packets are individually legitimate. Detection therefore relies on statistical features of whole *flows*: packet counts, sizes, inter-arrival times, TCP flags and rates. Those features are what machine-learning classifiers learn from.

Quantum machine learning (QML) asks whether quantum computers can help with such learning problems. One of its most studied ideas is the *quantum kernel method*. A classical data point $\boldsymbol{x}$ is encoded into a quantum state $|\phi(\boldsymbol{x})\rangle$ in a Hilbert space of dimension 2^n. The overlap $K(\boldsymbol{x}, \boldsymbol{x}') = |\langle\phi(\boldsymbol{x})|\phi(\boldsymbol{x}')\rangle|^2$ is then used as the kernel of an ordinary support vector machine. Havlíček et al. [2] proposed the *ZZ feature map*, an entangling circuit whose kernel is believed to be hard to evaluate classically for large n. That hardness is a prerequisite for a quantum advantage.

1.2 The project

This project applies quantum kernel SVMs (QSVMs) to network intrusion detection on the CIC-DDoS2019 dataset [3]. The accompanying paper [1] compares three methods:

1. an **Analytical kernel** built from single-qubit R_x rotations. This is a separable product state, so its kernel has the closed form $\prod_i \cos^2((b_i - a_i)/2)$;
2. a **Sampling kernel**, which estimates the same overlaps by running the circuit $U^\dagger(\boldsymbol{a})U(\boldsymbol{b})$ for 1024 shots, as real hardware would;
3. a **ZZ entangling kernel**, which uses the Havlíček feature map and exact statevector simulation.

All three are compared with a classical RBF-kernel SVM. The paper reports that the ZZ kernel wins in both settings: 98.79% vs. 96.97% in binary classification (benign vs. DDoS) and 92.00% vs. 80.00% in 6-class classification. It concludes that entanglement is necessary for a quantum advantage on this task.

1.3 Goal of this report

The task was to *build on the current state of the project*: improve the existing code without replacing it with something unrelated, then document the whole project and every change. The improved code keeps the same three modules (`data_loader.py`, `qsvm.py`, `main.py`), the same Cirq ZZ feature map class, the same CLI flags and the same dataset. We added two small modules (`preprocessing.py`, `evaluation.py`) and three supporting tools (`benchmark.py`, `make_figures.py`, `tests/`). The original code is preserved unchanged in `legacy/`, and `main.py -legacy` reproduces its protocol exactly.

The report has three parts:

- **Part I: the project as it was** (Sections 2–4). It covers the background theory (DDoS, SVMs, quantum computing, quantum kernels), the dataset as it actually exists in this repository, and a walk-through and audit of the original pipeline.

- **Part II: the improvements** (Sections 5–8). They are grouped into data integrity, computation, modelling, and evaluation/software. Each one comes with its motivation, the change, a derivation where needed, and how it was verified.
- **Part III: results and discussion** (Sections 9–10). This part re-runs the paper's comparison under the corrected protocol with 5 random seeds, adds ablations isolating each fix, and runs sweeps over bandwidth, qubit count, shots, circuit architecture, feature set and training-set size. It then discusses what the evidence says about quantum advantage on this task.

Summary of findings

- The data file contains 49,393 exact duplicate rows (65.9%). With duplicates kept, about 30% of test flows have an identical twin in the training set.
- The original kernel computation is exact but needed $2N^2$ circuit simulations. The new engine needs N and gives the same numbers to 10^{-7}, with speed-ups of 10^3–$10^5\times$ (Table 6).
- With 16 qubits and the original angle scale, the ZZ kernel is *concentrated*: the median off-diagonal entry is $\approx 2 \times 10^{-5}$ and the kernel matrix is close to the identity. A tuned bandwidth $\alpha < 1$ fixes this.
- Once the classical baseline gets the same preprocessing and the same tuning budget, the large QSVM advantages disappear. Section 9 gives the numbers.

2 Background

2.1 DDoS attacks in the dataset

CIC-DDoS2019 groups attacks into *reflection-based* attacks, where third-party servers answer spoofed requests and so hide and amplify the attacker, and *exploitation-based* attacks, which abuse protocol behaviour directly [3]. The five attack classes in this project's data file are summarised in Table 1.

Table 1: Attack classes present in `output.csv`.

Label	Type	Mechanism
`MSSQL`	reflection (TCP/UDP)	Spoofed requests to the SQL Server Resolution Protocol (UDP 1434). Servers answer with much larger instance listings.
`LDAP`	reflection (UDP)	Connectionless LDAP (CLDAP, UDP 389) queries whose responses are tens of times larger than the request.
`Portmap`	reflection (UDP)	Abuse of the RPC port mapper (rpcbind, UDP 111), which returns long service lists.
`UDP`	exploitation	High-rate flood of UDP datagrams to random ports. The victim spends resources answering with ICMP "port unreachable".
`Syn`	exploitation (TCP)	SYN flood: many half-open TCP handshakes exhaust the victim's connection table.
`BENIGN`	—	Normal background traffic generated by profiled user agents.

These classes leave different traces at the flow level. SYN floods produce very short flows with the SYN flag set and almost no backward traffic. Reflection attacks produce unidirectional flows with characteristic payload sizes. Benign traffic is far more varied. This is why flow statistics

separate benign from attack traffic so well, and also why the *reflection* classes (MSSQL, LDAP, Portmap) are the hard part of the multiclass problem: they look alike.

2.2 Flow-based detection and CICFlowMeter

A *flow* is the set of packets sharing a 5-tuple (source/destination IP and port, protocol) within a timeout. CICFlowMeter [4] turns packet captures into bidirectional flows and computes about 80 statistics per flow: durations, forward/backward packet counts and byte totals, packet-length moments, inter-arrival-time (IAT) moments, TCP flag counts, header lengths, bulk and sub-flow statistics, and active/idle periods. Two properties of these features matter a great deal later:

- **Heavy tails.** Rates such as `Flow Bytes/s` span nine orders of magnitude (Section 3).
- **Redundancy.** Many columns are deterministic functions of others (e.g. `Fwd Header Length.1` duplicates `Fwd Header Length`; `Packet Length Variance` is the square of `Packet Length Std`), and several are always zero.

2.3 Support vector machines and kernels

Given training data $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^{N}$ with $y_i \in \{-1, +1\}$, a soft-margin SVM [5] solves the dual problem

$$\max_{\boldsymbol{\beta}} \sum_i \beta_i - \frac{1}{2}\sum_{i,j} \beta_i \beta_j y_i y_j K(\boldsymbol{x}_i, \boldsymbol{x}_j) \quad \text{s.t.} \quad 0 \le \beta_i \le C, \;\; \sum_i \beta_i y_i = 0, \tag{1}$$

and predicts $f(\boldsymbol{x}) = \operatorname{sign}\big(\sum_i \beta_i y_i K(\boldsymbol{x}_i, \boldsymbol{x}) + b\big)$. The data enter *only* through the kernel $K(\boldsymbol{x}, \boldsymbol{x}') = \langle \varphi(\boldsymbol{x}), \varphi(\boldsymbol{x}') \rangle$, the inner product in some feature space. This is the *kernel trick*: any positive semi-definite (PSD) function can be plugged in, including one evaluated on a quantum computer. scikit-learn's `SVC(kernel='precomputed')` accepts the $N \times N$ Gram matrix $K_{ij} = K(\boldsymbol{x}_i, \boldsymbol{x}_j)$ for training and the $M \times N$ matrix $K(\boldsymbol{x}_m^{\text{test}}, \boldsymbol{x}_i)$ for prediction [7].

The RBF kernel. The classical baseline is $K_{\text{RBF}}(\boldsymbol{x}, \boldsymbol{x}') = \exp(-\gamma \|\boldsymbol{x} - \boldsymbol{x}'\|^2)$. The *bandwidth* γ sets the scale at which two points count as similar. If γ is too large, $K \to I$ and the SVM memorises the training set. If it is too small, $K \to \mathbf{1}\mathbf{1}^\top$ and every point looks the same. This idea carries over directly to quantum kernels (Section 2.6).

Regularisation C. C trades margin width against training errors. C and γ interact strongly, which is why they are normally tuned jointly by cross-validation.

Multiclass and probabilities. For k classes, `SVC` trains $k(k-1)/2$ one-vs-one classifiers and takes a vote. Probabilities (`predict_proba`) come from Platt scaling [6], a logistic model fitted on internal cross-validation folds. Platt scaling is randomised, so it must be seeded to be reproducible.

2.4 Quantum computing essentials

A qubit is a unit vector $|\psi\rangle = a\,|0\rangle + b\,|1\rangle \in \mathbb{C}^2$. An n-qubit register lives in $(\mathbb{C}^2)^{\otimes n} \cong \mathbb{C}^{2^n}$, with basis states $|z\rangle$ indexed by bit-strings $z \in \{0,1\}^n$. Gates are unitary matrices. The ones used here are

$$H = \tfrac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad R_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}, \quad R_x(\theta) = \begin{pmatrix} \cos\frac{\theta}{2} & -i\sin\frac{\theta}{2} \\ -i\sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix}, \tag{2}$$

and the two-qubit CNOT $|a, b\rangle = |a, a \oplus b\rangle$. Measuring in the computational basis returns z with probability $|\langle z|\psi\rangle|^2$ (the Born rule). A classical *statevector simulator* stores all 2^n complex amplitudes. With `complex64` this needs $8 \cdot 2^n$ bytes: 2 KB at 8 qubits, 512 KB at 16 and 8 MB at 20. A 58-qubit state would need about 2.3 exabytes, which is why the features are first reduced with PCA. The paper's figure of 128 GB for 58 qubits is a typo; $8 \cdot 2^{58}$ bytes $\approx 2.3 \times 10^{18}$ bytes.

Useful identities. Writing $s(z) = 1 - 2z \in \{+1, -1\}$ for the eigenvalue of Z on bit z, we have $R_z(\theta)|z\rangle = e^{-i\theta s(z)/2}|z\rangle$: R_z is *diagonal*. The sequence $\mathrm{CNOT}_{ij}\, R_z^{(j)}(\theta)\, \mathrm{CNOT}_{ij}$ maps $|z_i, z_j\rangle \mapsto e^{-i\theta s(z_i \oplus z_j)/2}|z_i, z_j\rangle$, and since $s(z_i \oplus z_j) = s(z_i)s(z_j)$,

$$\mathrm{CNOT}_{ij}\, R_z^{(j)}(\theta)\, \mathrm{CNOT}_{ij} = \exp\big(-i\tfrac{\theta}{2} Z_i Z_j\big), \tag{3}$$

which is also diagonal. This is the "ZZ interaction" of the feature map, and it is the key to the fast simulator in Section 6.

2.5 Quantum feature maps and kernels

A *feature map* is a parameterised circuit $U(\boldsymbol{x})$ that prepares $|\phi(\boldsymbol{x})\rangle = U(\boldsymbol{x})|0^n\rangle$. The *fidelity quantum kernel* is

$$K(\boldsymbol{a}, \boldsymbol{b}) = \big|\langle \phi(\boldsymbol{a})|\phi(\boldsymbol{b})\rangle\big|^2 = \big|\langle 0^n| U^\dagger(\boldsymbol{a}) U(\boldsymbol{b})|0^n\rangle\big|^2. \tag{4}$$

The second form shows how hardware computes it: prepare $U^\dagger(\boldsymbol{a})U(\boldsymbol{b})|0^n\rangle$, measure all qubits, and record how often the all-zeros string appears. K is a valid PSD kernel, because it is the inner product of the "density-matrix features" $\rho(\boldsymbol{x}) = |\phi(\boldsymbol{x})\rangle\langle\phi(\boldsymbol{x})|$ under the Hilbert–Schmidt inner product [10, 11]. It also satisfies $K(\boldsymbol{x}, \boldsymbol{x}) = 1$.

Separable R_x encoding (Method 1). With $|\phi(\boldsymbol{x})\rangle = \bigotimes_i R_x(x_i)|0\rangle$, the overlap factorises qubit by qubit. Since $R_x(a)^\dagger R_x(b) = R_x(b - a)$ and $\langle 0| R_x(\theta)|0\rangle = \cos(\theta/2)$,

$$K_{R_x}(\boldsymbol{a}, \boldsymbol{b}) = \prod_{i=1}^{n} \cos^2\Big(\frac{b_i - a_i}{2}\Big). \tag{5}$$

This is a classical, cheap-to-evaluate kernel. It is a product of one-dimensional kernels, and it closely resembles a periodic RBF kernel. No quantum computer is needed to evaluate it.

ZZ feature map (Method 3). Following [2], one repetition is

$$U_{\mathrm{ZZ}}(\boldsymbol{x}) = \underbrace{\prod_{(i,j)\in E} \exp\big(-i(\pi - x_i)(\pi - x_j) Z_i Z_j\big) \prod_i e^{-i x_i Z_i}}_{D(\boldsymbol{x})\ \text{(diagonal)}}\ H^{\otimes n}, \tag{6}$$

where E is the set of entangled pairs (adjacent pairs, *linear*, in the original code) and the whole block is repeated r times. The code implements $e^{-ix_iZ_i}$ as $R_z(2x_i)$ and the pair term as $\mathrm{CNOT} \cdot R_z(2(\pi - x_i)(\pi - x_j)) \cdot \mathrm{CNOT}$, using Eq. (3). Figure 1 shows one repetition on four qubits. For $r \geq 2$, the Hadamard layers sandwiched between diagonal layers are what is conjectured to make the kernel hard to compute classically [2].

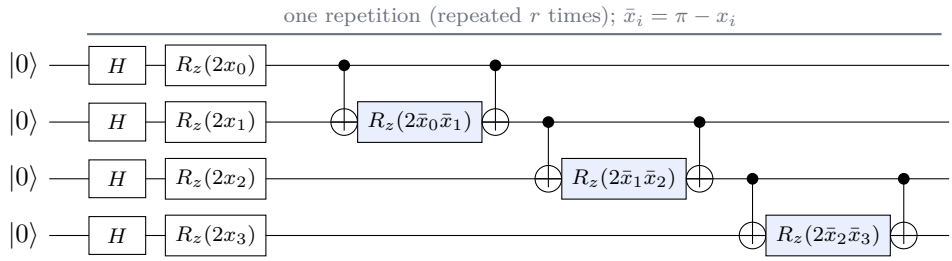


Figure 1: One repetition of the ZZ feature map with linear entanglement on 4 qubits. Every gate except H is diagonal in the computational basis.

2.6 Kernel concentration and bandwidth

Quantum kernels have a characteristic failure mode called *exponential concentration* [12]. For expressive feature maps on many qubits, the states of two different inputs are nearly orthogonal in the 2^n-dimensional space, so $K(\boldsymbol{a}, \boldsymbol{b}) \approx 0$ for all $\boldsymbol{a} \neq \boldsymbol{b}$ and the Gram matrix approaches the identity. An SVM trained on $K \approx I$ can fit the training set perfectly, but its decision function on new points is close to a constant. This is the quantum analogue of an RBF kernel with $\gamma \to \infty$. On hardware it is worse, because resolving values of order 2^{-n} needs exponentially many shots.

Shaydulin and Wild [13] and Canatar et al. [14] showed that *scaling the input*, $\boldsymbol{x} \mapsto \alpha\boldsymbol{x}$ with $\alpha < 1$, acts as a bandwidth: it moves the states closer together, restores off-diagonal mass, and can turn a kernel that does not generalise into one that does. The original project had no such parameter. Every feature used the full $[0, \pi]$ range ($\alpha = 1$).

2.7 When can a quantum kernel help?

A quantum kernel can only beat classical methods if (a) it cannot be computed efficiently classically, *and* (b) it fits the data better than the classical kernels one would actually use. Huang et al. [15] made (b) testable with the *geometric difference*

$$g(K_C \| K_Q) = \sqrt{\left\| \sqrt{K_Q}\,(K_C + \lambda I)^{-1} \sqrt{K_Q} \right\|_\infty}, \tag{7}$$

computed on the training Gram matrices, both normalised to trace N. If g is small (close to 1), then for *every* labelling the classical kernel generalises about as well as the quantum one, so no advantage is possible on this data. A potential advantage requires $g \gtrsim \sqrt{N}$. We compute g against the tuned RBF kernel for every run (Section 8). Recent large-scale benchmarking [17] also found that, under careful tuning, out-of-the-box classical models usually match or beat quantum models on small tasks. That is one more reason to give the classical baseline the same tuning budget as the quantum model.

3 The Dataset in This Repository

3.1 Where the data lives

The repository holds no folder of raw CIC-DDoS2019 CSVs. All data is in one 33 MB file, `output.csv`, which `main.py` reads through `data_loader.load_csv()` (default path `./output.csv`). It has 74,999 data lines and 88 columns: an index column (`Unnamed: 0`), seven identifier columns (`Flow ID`, source/destination IP and port, `Protocol`, `Timestamp`), 77 numeric CICFlowMeter features, `SimillarHTTP`, `Inbound` and `Label`.

Provenance. Several clues show how the file was made. Four of its lines repeat the header row (with `Label`="Label"), so it is a concatenation of several CICFlowMeter CSVs. The class names (`MSSQL`, `LDAP`, `Portmap`, `Syn`, `UDP`, without the `DrDoS_` prefix used in the January files) and the presence of `Portmap` match the CSVs of the *11 March* capture day, whose timestamps read `2018-11-03`. The paper attributes these classes to the 12 January "training day"; that should be corrected. Each attack class contributes roughly 10–20k rows (Table 2), and the `Unnamed: 0` index values (up to 482,780) point back to positions in much larger source files. Most importantly, each class covers a *very short time window*: all 14,999 `Syn` flows fall within 1.1 s, all `UDP` flows within 2 s, and all `LDAP` flows within about 2 minutes. The file is therefore a set of contiguous slices from each class's capture, not a random sample of the full dataset.

3.2 Class composition and duplicates

After removing the four repeated header lines, 74,995 rows remain. The loader counts two rows as duplicates when all 61 feature columns used by the paper and the label are equal. By that definition, **49,393 rows (65.9%) are exact duplicates** of an earlier row (Table 2).

Table 2: Rows, duplicates and unique rows per class in `output.csv` (paper feature set).

Class	Rows	Duplicates	Unique	Duplicate share
BENIGN	5,126	786	4,340	15.3%
LDAP	9,931	9,353	578	94.2%
MSSQL	19,657	13,697	5,960	69.7%
Portmap	10,284	9,521	763	92.6%
Syn	14,999	8,706	6,293	58.0%
UDP	14,998	7,330	7,668	48.9%
Total	74,995	49,393	25,602	65.9%

The reflection attacks are almost entirely repetitive: only 578 of 9,931 LDAP flows and 763 of 10,284 Portmap flows are distinct. This is expected, since an amplification attack replays the same request millions of times and CICFlowMeter maps identical packet trains to identical feature vectors. It has a direct consequence for evaluation. If duplicates are kept and the data is split at random, a large share of test flows have an identical copy in the training set, and the classifier only needs to look them up. Section 5 measures this effect: about 30% of test rows.

The low number of unique LDAP and Portmap flows also caps the balanced multiclass sample size. With deduplication, the smallest class (LDAP, 578 unique flows) limits a balanced 6-class sample to at most $6 \times 578 = 3468$ rows.

3.3 Features

The original loader lists 61 numeric columns (the "paper" feature set). Twelve are constant zero in this file: `Bwd PSH Flags`, `Fwd/Bwd URG Flags`, `FIN`, `PSH` and `ECE Flag Count`, and all six `Bulk` statistics. That leaves **49 informative features**, not the 58 stated in the paper. The original loader padded the matrix with zero columns to reach exactly 58; those columns carry no information and are dropped by PCA anyway. The "all" feature set adds 16 more columns (Subflow counts, `Init_Win_bytes_forward/backward`, `act_data_pkt_fwd`, `min_seg_size_forward`, Active/Idle statistics). Identifier columns (IPs, ports, timestamps, flow IDs) are never used: in this testbed they would reveal the attack hosts and time windows and give a trivially perfect but meaningless classifier.

The rate columns `Flow Bytes/s` and `Flow Packets/s` contain $\pm\infty$ in about 2,650 rows (CICFlowMeter divides by a zero duration). Five rows are NaN. Both are set to 0.

Heavy tails. Most features are extremely skewed. `Flow Bytes/s` ranges up to 2.9×10^9. Under linear min–max scaling to $[0, \pi]$, 35% of flows receive an angle below 0.01 rad (Figure 2). Across all 49 informative features, the median of the 90th percentile after min–max scaling is only 0.0017: for 69% of the features, nine out of ten flows end up in the bottom 5% of the rotation range. Section 5 explains why this matters for angle encoding and how a signed log transform fixes it.

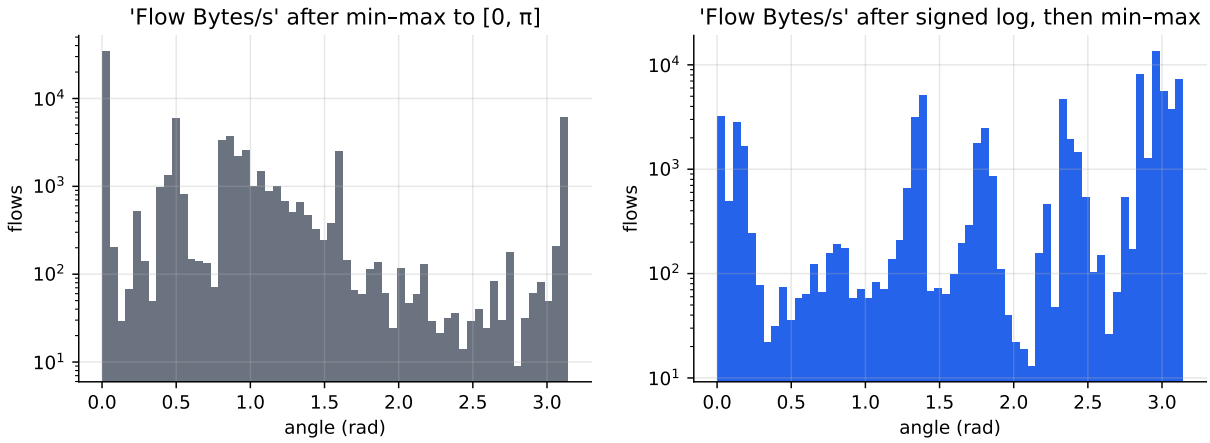


Figure 2: Distribution of the rotation angle derived from `Flow Bytes/s` (log-scale counts). Left: linear min–max, as in the original code; almost all mass sits at angle ≈ 0. Right: after a signed $\log(1 + |x|)$ transform the full $[0, \pi]$ range is used.

3.4 Tasks

- **Binary**: `BENIGN` (label 0) vs. any attack (label 1, "DDoS"). Classes are balanced by undersampling. With deduplication the attack half is drawn from the pooled unique attack flows, which are dominated by UDP, Syn and MSSQL.
- **Multiclass**: the six labels, BENIGN, LDAP, MSSQL, Portmap, Syn and UDP (alphabetical codes 0–5), undersampled to equal counts.

Both tasks use `-samples` (default 2000) as the total sample size, split 80/20 into train/test with stratification. In multiclass mode that means 333 samples per class, 1,598 training samples and 400 test samples.

4 The Original Pipeline and Its Audit

4.1 Architecture

The original code (preserved verbatim in `legacy/`) has three modules. Figure 3 shows the data flow.

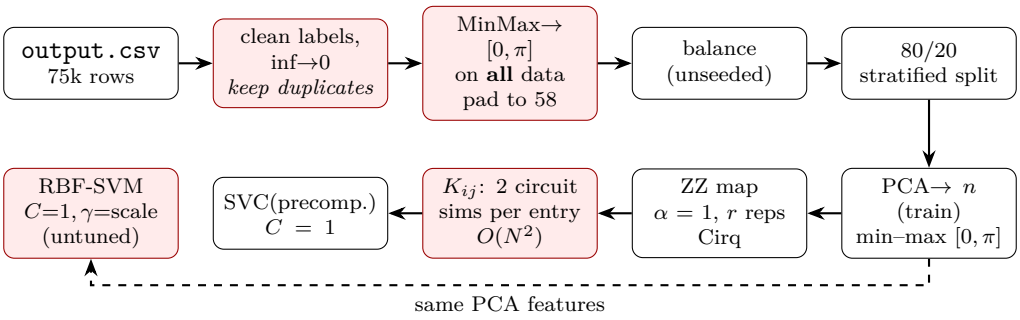


Figure 3: Original pipeline. Red boxes mark steps changed by this work.

`data_loader.py`. `load_csv()` reads the CSV, strips column names, removes repeated header rows, and draws up to 3×`max_samples` rows at random (`random_state=42`). It builds binary or multiclass labels, selects up to 61 named feature columns, replaces $\pm\infty$/NaN by 0, drops zero-variance columns, and pads or trims to exactly 58 columns. It then undersamples each class to min(smallest class, max_samples/k) using the *unseeded* global `np.random`. Finally it fits a `MinMaxScaler` to $[0, \pi]$ on *all* remaining rows and returns them.

`qsvm.py`. `ZZFeatureMap.build_encoding_circuit()` builds the circuit of Figure 1 in Cirq. `build_kernel_circuit()` builds $U^\dagger(\boldsymbol{a})U(\boldsymbol{b})$ plus measurement, for the sampling mode. `QuantumKernel._kernel_statevector(a,b)` simulates *both* circuits with `cirq.Simulator` and returns $|\langle\psi_a|\psi_b\rangle|^2$. `compute_kernel_matrix()` loops over all (i, j) pairs, using symmetry for the training matrix. `QSVM.fit()` fits PCA to `n_qubits` components on the training rows, rescales each component to $[0, \pi]$ with training min/max, computes K_{train} and fits `SVC(kernel='precomputed', C=1, probability=True)`. `predict()` and `predict_proba()` *each* rebuild the full test kernel.

`main.py`. `run_pipeline()` loads the data, splits it 80/20 (stratified, `random_state=42`), truncates the training set to 3000 rows, trains the QSVM, then trains `SVC(kernel='rbf', C=1, gamma='scale')` on standardised PCA features (same projection) and prints accuracy, a classification report, timings and a 4-panel figure (two confusion matrices, ROC, accuracy bars).

4.2 Reported results

Table 3 reproduces the paper's headline numbers. Only the ZZ method exists in the code. The Analytical and Sampling variants were run from earlier versions that are not in the repository.

Table 3: Results reported in the paper [1] (single run each).

Method	Mode	QSVM acc.	SVM acc.	QSVM AUC	N_{test}
Analytical (R_x, 16q)	binary	97.57%	98.54%	0.987	1028
Analytical (R_x, 16q)	multiclass	89.63%	87.68%	–	3084
Sampling (1024 shots)	binary	98.54%	98.54%	0.992	1028
Sampling (1024 shots)	multiclass	89.63%	87.68%	–	3084
ZZ (8q, 2 reps)	binary	**98.79%**	96.97%	0.999	822
ZZ (16q, 1 rep)	multiclass	**92.00%**	80.00%	–	996

4.3 Audit findings

We read every line of the three modules and checked the data file. Table 4 lists the issues found, ordered roughly by how much they can change the conclusions.

Table 4: Issues found in the original pipeline.

#	Issue	Effect	Severity
I1	Exact duplicates kept	65.9% of rows are duplicates; ≈30% of test flows have a training twin (measured). Accuracy is inflated by memorisation.	High
I2	Untuned, unequal baselines	QSVM and RBF both fixed at $C = 1$; RBF γ fixed at `'scale'`. The comparison measures default settings, not models.	High
I3	Kernel concentration	16-qubit ZZ kernel with $\alpha = 1$: median off-diagonal entry $\approx 2 \times 10^{-5}$, 84–89% of entries below 10^{-3}, so $K \approx I$ (measured in Section 9). No bandwidth control.	High
I4	Cost $2N^2$ simulations	Two full circuit simulations per kernel entry, and the test kernel is computed twice. A 1600-sample, 16-qubit run is estimated at ≈44 h.	High (practical)
I5	Global MinMax before split	Test-set min/max leak into the training features.	Medium
I6	Linear encoding of heavy tails	For most features, 90% of flows occupy $< 5\%$ of $[0, \pi]$, so qubits receive near-constant angles.	Medium
I7	Single run, no significance	Differences of 1–2 points on 400–1000 test flows are within sampling noise. There are no error bars and no tests.	Medium
I8	Non-reproducible sampling	Class balancing uses unseeded `np.random`; Platt scaling is unseeded. Each run draws different data.	Medium
I9	Inconsistent sample sizes	The paper compares methods on different N (822 vs. 1028 test).	Medium
I10	Missing methods	The Analytical and Sampling kernels in the paper are not in the code.	Medium
I11	Narrow metrics	Accuracy only. IDS operators care about false-alarm rate and detection rate. Multiclass AUC is not reported.	Low
I12	Cosmetic	Zero-padding to 58 columns; hard-coded "58 →" message; silent truncation to 3000 training rows.	Low
I13	Pre-sampling shrinks the data	3×`max_samples` random rows are drawn *before* balancing. BENIGN is only 6.8% of the file, so `-samples 2000` yields 411 BENIGN rows and 822 binary samples in total (165 test flows).	Medium

How small the binary test set was. Running the original loader confirms I13: with the default `-samples 2000` it returns exactly 822 binary samples, the N quoted in the paper for the ZZ binary experiment, so the test split had 165 flows. The reported 98.79% vs. 96.97% is then 163/165 vs. 160/165, a difference of **three test flows**. Even if all three were cases where only the QSVM was right, the exact McNemar test would give $p = 0.25$. The multiclass figures (92.00% vs. 80.00%, $N = 996 = 6 \times 166$) correspond to a 200-flow test set. The 12-point gap is large, but

it comes from a single split with the issues above.

None of these issues is unusual. Duplicate leakage in particular is a well-documented trap in intrusion-detection benchmarks [22, 21], and untuned baselines are the most common cause of overstated quantum advantage [17]. The core idea and the quantum code itself are sound. The fast engine reproduces the original kernel values exactly (Section 6.4).

5 Improvements I: Data Integrity

Figure 4 shows the improved pipeline. Table 5 maps every audit finding to its fix and the file that implements it. The next four sections describe the changes in detail.

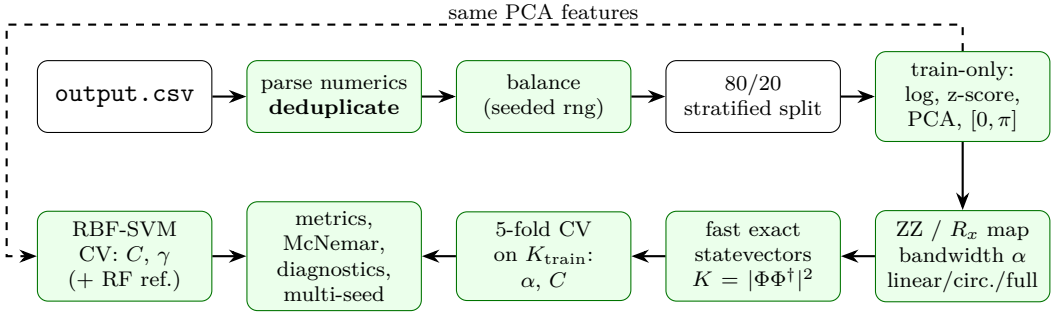


Figure 4: Improved pipeline. Green boxes are new or changed. Every fitted statistic comes from the training split only.

Table 5: Audit finding → change → location.

#	Change	Where
I1	Drop exact duplicate (features, label) rows before sampling; report the train/test overlap	`data_loader.py`
I2	5-fold CV grid search on the training set for QSVM (α, C) and RBF (C, γ), same metric and folds	`evaluation.py`
I3	Bandwidth α (input scaling) in all feature maps; concentration diagnostics	`qsvm.py`, `evaluation.py`
I4	One simulation per sample plus a Gram product; fast exact NumPy engine; state cache	`qsvm.py`
I5	All scaling/PCA fitted on training rows only; test values clipped to $[0, \pi]$	`preprocessing.py`
I6	Signed $\log(1 + \lvert x \rvert)$ before standardisation and PCA	`preprocessing.py`
I7	`-seeds`: repeat over seeds, report mean ± std; exact McNemar test per seed	`main.py`, `evaluation.py`
I8	Seeded `default_rng`; seeded Platt scaling, PCA and CV	all modules
I9	All methods share one loader, one split and one preprocessing per seed	`main.py`, `benchmark.py`
I10	`-kernel rx` (closed form) and `-sampling` (shots) added to the pipeline	`qsvm.py`
I11	Balanced accuracy, macro-F1, MCC, AUC (binary and OvR), false-alarm and detection rates	`evaluation.py`
I12	No padding; real feature counts reported; no silent truncation	`data_loader.py`, `main.py`
I13	Balance from the full (deduplicated) pool, so `-samples` is the real sample size	`data_loader.py`

5.1 Duplicate removal (I1)

Problem. With duplicates kept, many test flows are byte-identical to training flows. `data_loader.duplicate_overlap()` measures this directly by hashing every training row and counting test rows found in the set. For the original protocol it reports **28–30%** of test rows in the binary task (Table 8). A kernel classifier gets those rows right automatically: $K(\boldsymbol{x}, \boldsymbol{x}) = 1$ is the largest possible kernel value, so the training twin dominates the decision function.

Change. `load_csv(dedup=True)` (the default) removes rows whose 61 feature values *and* task label equal those of an earlier row, before any sampling:

```
key = X_df.copy(); key["__label"] = y_all
keep = ~key.duplicated().values
```

The key uses the label of the task (`y_all`), not the raw attack name: in binary mode an LDAP flow and an identical Portmap flow are both "DDoS", so the second one is a duplicate (175 extra rows). Identical feature vectors with *different* task labels (177 rows in multiclass mode, 2 in binary mode) are kept. They are genuine ambiguity, not leakage. `-keep-duplicates` restores the old behaviour for comparison. After deduplication the overlap falls to 0–0.25% (a handful of rows that differ only in dropped zero-variance columns).

Why it matters for the conclusions. Duplicates do not affect all models equally. A kernel that is very sharp ($K \approx I$ off the diagonal, as with the untuned 16-qubit ZZ map) behaves almost like a nearest-neighbour lookup. It benefits *most* from exact twins and is hurt most on genuinely new flows. Leakage can therefore create a spurious advantage for exactly the most "quantum" kernel.

5.2 Leakage-free preprocessing (I5)

Problem. The original loader called `MinMaxScaler().fit_transform(X)` on all sampled rows, then `main.py` split them. Each feature's minimum and maximum therefore included test rows. With heavy-tailed features the maximum is often a single extreme flow, so whether that flow lands in train or test changes the scale of every training feature.

Change. `data_loader.py` now returns raw, unscaled features. A new class, `QuantumPreprocessor` in `preprocessing.py`, is fitted on the training split only, inside `QSVM.fit()`. Its `transform()` applies the stored statistics to the test set and clips out-of-range angles to $[0, \pi]$. The classical baselines use `transform_pca()` of the *same* fitted object, so both models see identical information. The original behaviour is kept as `LegacyPreprocessor` plus `legacy_global_scale()`, selected with `-legacy-scaling`.

5.3 Signed log transform (I6)

Problem. Angle encoding maps each feature to a rotation. If 90% of flows fall within 0.002 rad of each other, the corresponding qubit is in essentially the same state for all of them, and the kernel cannot tell them apart along that direction. PCA does not help, because it is dominated by the few extreme flows that define the variance.

Change. Before standardisation and PCA, every feature is transformed by

$$x \mapsto \operatorname{sign}(x)\, \log(1 + |x|). \tag{8}$$

This is monotone, defined for zero and negative values (`Fwd Header Length` is negative in 12,592 rows, an artefact of CICFlowMeter), and compresses nine orders of magnitude into about 22 units. Figure 2 shows the result for `Flow Bytes/s`: the median angle moves from 0.45 to 2.86 rad and the fraction of flows below 0.01 rad drops from 35% to 4%. The pipeline is then: drop zero-variance columns → signed log → `StandardScaler` → `PCA`(n) → per-component min–max to $[0, \pi]$, all fitted on training data. `-no-log` disables the transform for ablation.

5.4 Reproducible sampling and real sample sizes (I8, I12, I13)

All randomness now flows from the `-seeds` values: `np.random.default_rng(seed)` for balancing, `random_state=seed` for the split, PCA, `StratifiedKFold`, Platt scaling, the Random Forest and the shot-noise generator. Two runs with the same seed produce byte-identical results; this is verified in `tests/test_core.py::test_dedup_and_reproducible`. Classes are balanced by drawing from the whole (deduplicated) pool of each class, so `-samples 2000` now really gives 2000 samples (1000 per class in binary mode, 333 per class in multiclass mode) instead of 822. `-legacy` uses the same sampler, so that the original and improved protocols are always compared on the same number of flows. Numeric parsing uses `pd.to_numeric(errors='coerce')` so stray strings cannot crash the cast. Zero padding to 58 columns was removed, and the true number of features entering PCA is printed.

6 Improvements II: Computation

The original kernel code is correct but very slow. It is so slow that it ruled out the experiments needed to answer the project's own research question: repeated runs, tuning, ablations, and realistic training-set sizes at 16 qubits. This section describes three changes that keep the exact same kernel values while removing the bottleneck.

6.1 Simulate each sample once (statevector caching)

Problem. `_kernel_statevector(a, b)` simulated $U(\boldsymbol{a})\,|0\rangle$ and $U(\boldsymbol{b})\,|0\rangle$ from scratch for every pair. For N training and M test samples the original code performed

$$\underbrace{2 \cdot \tfrac{N(N+1)}{2}}_{K_{\text{train}}} + \underbrace{2 \cdot 2MN}_{K_{\text{test}}\ \text{in predict and predict_proba}} \tag{9}$$

circuit simulations: about 5.1 million for $N = 1600$, $M = 400$. Yet $|\phi(\boldsymbol{x})\rangle$ depends only on $\boldsymbol{x}$.

Change. `QuantumKernel.states(X)` returns the matrix $\Phi_X \in \mathbb{C}^{N \times 2^n}$ whose rows are the feature states. The whole kernel matrix is then one matrix product:

$$K(X, Y) = \left| \overline{\Phi_X}\, \Phi_Y^{\top} \right|^{\circ 2} \qquad \text{(element-wise squared modulus).} \tag{10}$$

This needs $N + M$ state preparations and one complex GEMM (`cgemm` in OpenBLAS, multi-threaded), computed in row blocks of 512 to bound memory. The most recent two state matrices are memoised. When the test kernel is built, the training states are reused rather than re-simulated, and `QSVM.predict`, `predict_proba` and `decision_function` share one cached test kernel. With the Cirq backend this alone cuts the 16-qubit cost from about 44 hours to about a minute (Table 6).

6.2 An exact fast engine for the ZZ feature map

Cirq is a general-purpose simulator. It applies each of the $\approx 4n$ gates per repetition as a separate tensor contraction, with Python overhead per gate and per circuit. The ZZ map has special structure that a dedicated engine can use.

Proposition 1. *Let* $s_i(z) = 1 - 2z_i$ *and*

$$\Phi_{\boldsymbol{x}}(z) = \sum_{i=1}^{n} x_i\, s_i(z) + \sum_{(i,j)\in E} (\pi - x_i)(\pi - x_j)\, s_i(z) s_j(z). \tag{11}$$

Then one repetition of the ZZ map is $U = D_{\boldsymbol{x}} H^{\otimes n}$ *with* $D_{\boldsymbol{x}} |z\rangle = e^{-i\Phi_{\boldsymbol{x}}(z)} |z\rangle$*, and the feature state after* r *repetitions is*

$$|\phi_r(\boldsymbol{x})\rangle = D_{\boldsymbol{x}} H^{\otimes n} \cdots D_{\boldsymbol{x}} H^{\otimes n} |0^n\rangle\,, \qquad |\phi_1(\boldsymbol{x})\rangle = 2^{-n/2} \sum_z e^{-i\Phi_{\boldsymbol{x}}(z)} |z\rangle\,. \tag{12}$$

Proof. $R_z(2x_i)$ acts on $|z\rangle$ as $e^{-ix_i s_i(z)}$. By Eq. (3), the gate block for pair (i,j) acts as $e^{-i(\pi-x_i)(\pi-x_j)s_i(z)s_j(z)}$. All these operators are diagonal, so they commute and their phases add, giving $D_{\boldsymbol{x}}$. Finally, $H^{\otimes n} |0^n\rangle = 2^{-n/2} \sum_z |z\rangle$. □

Implementation (`FastZZStatevector` in `qsvm.py`).

1. **Phase table.** Precompute the sign matrix $S \in \{\pm 1\}^{2^n \times n}$, with qubit 0 as the most significant bit to match Cirq's ordering, and the pair-product matrix $S_P \in \{\pm 1\}^{2^n \times |E|}$. For a batch of inputs X (rows $\boldsymbol{x}$), all phases are two real matrix products: $\Phi = XS^\top + WS_P^\top$, where $W_{b,(i,j)} = (\pi - x_{b,i})(\pi - x_{b,j})$.

2. **First repetition** is free: $\psi = 2^{-n/2} e^{-i\Phi}$, element-wise.

3. **Hadamard layers.** $H^{\otimes n} = H^{\otimes n_1} \otimes H^{\otimes n_2}$ with $n_1 = \lfloor n/2 \rfloor$. Reshaping ψ into a $2^{n_1} \times 2^{n_2}$ matrix M (row-major) gives $(A \otimes B)\,\mathrm{vec}(M) = \mathrm{vec}(AMB^\top)$. Each layer is therefore two small batched matrix multiplications with 256×256 matrices at $n = 16$, instead of a 65536×65536 operator or 16 strided butterfly passes.

4. Multiply by $D_{\boldsymbol{x}}$ again and repeat.

The cost per sample is $O(2^n(n + |E|))$ for the phases and $O(2^n(2^{n_1} + 2^{n_2}))$ per Hadamard layer. Everything is vectorised over batches of 128 samples. States are stored as `complex64`, the same precision Cirq uses by default.

Algorithm 1 Training kernel with the fast engine

input: angles $A \in [0,\pi]^{N\times n}$, bandwidth α, reps r, pairs E
$X \leftarrow \alpha A$
for each batch B of 128 rows of X **do**
  $\Phi \leftarrow XS^\top + W(X)S_P^\top$; $D \leftarrow e^{-i\Phi}$
  $\psi \leftarrow 2^{-n/2} D$
  for $k = 2$ **to** r **do**
    $\psi \leftarrow \mathrm{vec}\big(H^{\otimes n_1}\,\mathrm{mat}(\psi)\,(H^{\otimes n_2})^\top\big) \odot D$
  end for
  store ψ as rows of Φ_X
end for
$K \leftarrow |\overline{\Phi_X}\Phi_X^\top|^{\circ 2}$; symmetrise; set $\mathrm{diag}(K) = 1$

6.3 Other kernels on the same engine

Separable R_x kernel. With the fast backend, Eq. (5) is evaluated in closed form with broadcasting, in row blocks of 256. With the Cirq backend the product state is simulated, which serves as a cross-check.

Finite shots. Running $U^\dagger(\boldsymbol{a})U(\boldsymbol{b})$ for s shots and counting the all-zero outcomes gives a count distributed exactly as $\text{Binomial}(s, K(\boldsymbol{a}, \boldsymbol{b}))$: each shot is an independent Bernoulli trial with success probability $|\langle 0^n| U^\dagger(\boldsymbol{a})U(\boldsymbol{b}) |0^n\rangle|^2 = K(\boldsymbol{a}, \boldsymbol{b})$. The fast backend therefore computes the exact kernel and draws

$$\hat{K}_{ij} = \frac{1}{s}\,\text{Binomial}(s, K_{ij}), \tag{13}$$

independently for each upper-triangular entry, then mirrors it. This has *the same distribution* as the original sampling method at a tiny fraction of the cost. The real circuit path is still available with `-backend cirq -sampling`. The estimator is unbiased (verified in `test_shot_noise_is_unbiased`). The resulting matrix need not be PSD; libsvm tolerates mildly indefinite kernels.

6.4 Verification

Speed is only useful if the numbers are unchanged. `tests/test_core.py` checks:

- fast statevectors equal Cirq's (`atol` 10^{-5}) for linear, circular and full entanglement, 2 and 3 repetitions, $\alpha = 1$ and $\alpha = 0.3$, and for the R_x map. Observed maximum deviation: 2.4×10^{-7}, i.e. `float32` rounding;
- the new kernel value equals the value from the *original, unmodified* `legacy/qsvm.py` (`QuantumKernel(6,2).compute_kernel_value`) to 5 decimals;
- Gram matrices are symmetric with unit diagonal, and their smallest eigenvalue is $> -10^{-5}$ (PSD up to rounding).

`main.py -backend cirq` and `-backend fast` produce identical predictions (TESTING.md §6).

6.5 Speed

Table 6 and Figure 5 compare the cost of the training (1600^2) plus test (400×1600) kernels. "Original" is extrapolated from the measured per-entry time of `legacy/qsvm.py` times the number of entries it evaluates. "Cirq cached" is the measured per-state Cirq time times 2000 states, plus the measured Gram-product time. "Fast" is measured end to end.

Table 6: Kernel computation cost, $N_{\text{train}} = 1600$, $N_{\text{test}} = 400$, 2 repetitions, linear entanglement (8-core CPU). Legacy sampling uses 1024 shots per entry.

Qubits	Original/entry	Original total	Orig. sampling	Cirq cached	Fast	Speed-up
4	8.6 ms	6.1 h	8.1 h	8.5 s	0.06 s	344,270×
8	24.7 ms	17.6 h	16.9 h	17.3 s	0.16 s	392,457×
12	34.3 ms	24.4 h	–	27.7 s	0.88 s	99,298×
16	61.6 ms	43.8 h	–	67.5 s	26.41 s	5,969×

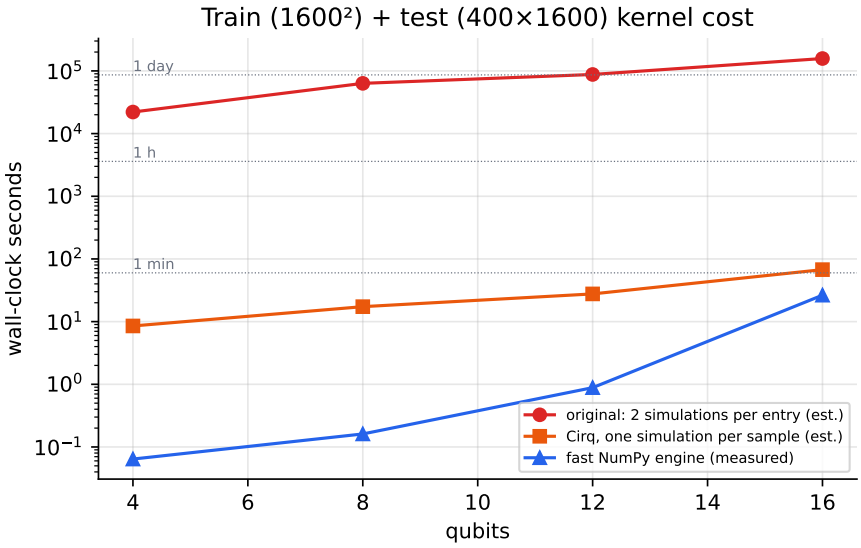


Figure 5: Wall-clock cost of a full train+test kernel (log scale). Original and Cirq-cached values are estimates from measured unit costs; fast-engine values are measured end to end.

At 16 qubits the fast engine is limited by memory bandwidth: 2000 states × 65,536 amplitudes × 8 bytes is about 1 GB, and the Gram product is about 1.7×10^{11} complex multiply-adds. It is still roughly 6,000 times faster than the original, which is what makes the 5-seed, cross-validated experiments in Section 9 feasible.

7 Improvements III: Modelling

7.1 Kernel bandwidth α (I3)

Problem. With n qubits and inputs spread over $[0, \pi]$, the ZZ feature states of two different flows are nearly orthogonal. Roughly, each qubit and each pair contributes an independent "random" phase, and the overlap of two random states in dimension 2^n is of order 2^{-n}. The experiments confirm this: at 16 qubits with $\alpha = 1$ and 2 repetitions, the *median* off-diagonal training kernel entry is about 2×10^{-5} and 84–89% of entries are below 10^{-3}. The mean is higher (about 0.03, Table 10) only because a few near-identical flows still overlap strongly. An SVM can still *separate* the training set with such a kernel, but each test prediction is then decided by a handful of nearest training points, and the kernel's geometry carries little information.

Change. All feature maps now take a bandwidth parameter. Angles from preprocessing are multiplied by α before encoding:

$$|\phi_\alpha(\boldsymbol{\theta})\rangle = U(\alpha\boldsymbol{\theta})\,|0^n\rangle\,, \qquad \boldsymbol{\theta} \in [0, \pi]^n. \tag{14}$$

Small α moves all states towards $U(\mathbf{0})\,|0^n\rangle$, raising off-diagonal kernel values. In the limit $\alpha \to 0$ the kernel tends to the constant 1. For the separable R_x map, α is exactly an RBF-like length-scale: $\cos^2(\alpha\Delta/2) \approx e^{-\alpha^2\Delta^2/4}$ for small $\alpha\Delta$. For the ZZ map, the pair term $(\pi - \alpha\theta_i)(\pi - \alpha\theta_j) = \pi^2 - \alpha\pi(\theta_i + \theta_j) + \alpha^2\theta_i\theta_j$ shows that α also controls how strongly the *product* (entangling) terms depend on the data, relative to the linear ones. $\alpha = 1$ reproduces the original circuit exactly (`ZZFeatureMap` and the fast engine both take `alpha`).

7.2 Cross-validated tuning with equal budgets (I2)

Problem. With $C = 1$ and a fixed scale for both models, the comparison measures how well two *default settings* happen to suit this dataset. RBF-SVM accuracy is very sensitive to γ and C. The default `gamma='scale'` ($1/(d \cdot \mathrm{Var}(X))$) is a heuristic, not an optimum.

Change. `evaluation.tune_quantum()` performs a grid search over $\alpha \in \{0.05, 0.1, 0.2, 0.4, 1.0\}$ and $C \in \{0.1, 1, 10, 100\}$, scoring *macro-F1* by 5-fold stratified CV *on the training split only*. The expensive part, the kernel, is computed once per α. Every C and every fold then reuses sub-blocks $K[\mathrm{tr}, \mathrm{tr}]$ and $K[\mathrm{va}, \mathrm{tr}]$ of that one matrix, so the search costs 5 kernel computations rather than 100 fits' worth. `tune_rbf()` runs `GridSearchCV` for the RBF-SVM over the same C grid and $\gamma \in \{\mathrm{scale}, 0.01, 0.03, 0.1, 0.3, 1\}$, with the same folds, metric and seed. The best configuration is refitted on the full training split and evaluated once on the test set. Both models get 20–24 candidate configurations: an equal budget.

```
alpha, C, K_train, table = tune_quantum(
    lambda a: make_kernel(a).compute_kernel_matrix(A_train, verbose=False),
    y_train, args.alphas, args.Cs, seed=seed)
```

`-no-tune -alpha A -C C` fixes the values instead. `-alphas` and `-Cs` change the grids.

7.3 Entanglement topology and depth

`entangling_pairs(n, entanglement)` generalises the hard-coded adjacent pairs: `linear` $\{(i, i{+}1)\}$ (original, $n-1$ pairs), `circular` (adds $(n{-}1, 0)$), and `full` (all $\binom{n}{2}$ pairs, the default in Qiskit's `ZZFeatureMap`). Both the Cirq circuit builder and the fast engine use the same pair list, and `-n-reps` sets the depth. These options answer the paper's "higher-order entanglement" future-work item within the existing circuit family.

7.4 All three paper methods in one pipeline (I10)

`-kernel rx` selects the separable R_x map (Method 1), and `-sampling -shots s` adds finite-shot estimation to any kernel (Method 2). Together with the default ZZ map (Method 3), all methods now share one loader, one split per seed, one preprocessing object and one tuning protocol. The paper's comparisons used different N for different methods (822 vs. 1028 binary test samples); here every method sees exactly the same flows.

7.5 Stronger classical references

Besides the tuned RBF-SVM on the *same* PCA features, which isolates the effect of the kernel, `-extra-baselines` trains a Random Forest [23] (300 trees) on *all* raw features without PCA. This is a typical practical NIDS baseline. It answers a different question: how much accuracy is left on the table by compressing to n components at all?

8 Improvements IV: Evaluation and Software

8.1 Metrics (I11)

`evaluation.classification_metrics()` reports, for every model and seed:

- **Accuracy** and **balanced accuracy** (mean per-class recall).
- **Macro-F1**: the unweighted mean of per-class F1. It is the CV selection criterion, because it penalises neglecting any class.
- **MCC** (Matthews correlation): a single-number summary that stays informative under imbalance.
- **ROC-AUC**. In binary mode it is computed from `decision_function`, the raw SVM margin, rather than from Platt probabilities, so it reflects the model and not the calibration fold. In multiclass mode it is the macro one-vs-rest AUC from `predict_proba`. The paper reported no multiclass AUC.
- **False-alarm rate** (FAR): the fraction of BENIGN flows flagged as any attack. **Detection rate** (DR): the fraction of attack flows flagged as *some* attack. These are the two numbers an IDS operator cares about. In multiclass mode DR ignores confusions between attack types, since an LDAP flow labelled MSSQL is still blocked.

Confusion matrices are stored for every seed.

8.2 Repetition and significance (I7)

Multiple seeds. `-seeds 0 1 2 3 4` repeats the *whole* pipeline (sampling, split, preprocessing, tuning, test) with five different seeds and reports mean ± standard deviation. With 400 test flows, one misclassification is 0.25 points, and the seed-to-seed standard deviation is typically 0.2–1 point. Single-run differences of that size cannot be interpreted.

McNemar's test. For each seed, the QSVM and RBF-SVM predictions on the *same* test flows are compared with the exact McNemar test [19, 20]. Let n_{01} be the number of flows only the QSVM gets right and n_{10} the number only the RBF gets right. Under H_0 (equal error rates), $n_{01} \sim \text{Binomial}(n_{01} + n_{10}, \frac{1}{2})$, and the two-sided exact binomial p-value is reported. Tables in Section 9 give the number of seeds (out of 5) with $p < 0.05$.

8.3 Kernel diagnostics

For every run, `kernel_diagnostics()` records, on the training Gram matrix:

- **Concentration**: mean and standard deviation of the off-diagonal entries. Values near 0 with tiny spread indicate $K \approx I$.
- **Centred kernel–target alignment** [18]:

$$A(K, Y) = \frac{\langle K_c, T_c \rangle_F}{\|K_c\|_F \|T_c\|_F}, \qquad K_c = HKH,\ T = YY^\top,\ H = I - \tfrac{1}{N}\mathbf{1}\mathbf{1}^\top, \tag{15}$$

  where Y is the one-hot label matrix. Alignment measures how well kernel similarity matches label similarity. It is computed for both the quantum and the tuned RBF kernel.
- **Geometric difference** $g(K_{\text{RBF}} \| K_Q)$ (Eq. 7) on a random subset of 400 training points, with $\lambda = 10^{-3}$, both kernels normalised to trace N.

8.4 Software engineering

Backward compatibility. Every original CLI flag (`-csv`, `-samples`, `-mode`, `-sampling`, `-n-qubits`, `-n-reps`) keeps its meaning. The public classes (`ZZFeatureMap`, `QuantumKernel`, `QSVM`) keep their names and methods, including `compute_kernel_value`, `compute_kernel_matrix`, `build_kernel_circuit`, `predict_proba` and `score`, plus a `QSVM.pca` property. New behaviour is opt-out (`-legacy`, `-legacy-scaling`, `-keep-duplicates`, `-no-log`, `-no-tune`).

New CLI options. `-kernel`, `-entanglement`, `-backend`, `-alpha`, `-C`, `-alphas`, `-Cs`, `-shots`, `-seeds`, `-features`, `-extra-baselines`, `-out-dir`, `-quiet`. See Appendix A.

Outputs. Each run writes `results.json`, containing all arguments, per-seed metrics, tuning tables, diagnostics, McNemar results and data bookkeeping (duplicates removed, overlap), plus a 6-panel figure `results_seed<k>.png` (Figure 6).

Tests. `tests/test_core.py` contains 19 `unittest` cases (standard library only, since pytest is not installed in the venv), covering engine correctness against Cirq and against the original code, kernel properties, leakage-freedom of preprocessing, loader reproducibility and deduplication, the metric implementations, and an end-to-end toy problem. They run in about 15 s.

Benchmark suite. `benchmark.py` defines nine experiments (speed, headline, ablation, bandwidth, qubits, shots, architecture, features, samples). Each configuration is a call to `main.main()` with a fixed argument list, its results go to `outputs/benchmark/<experiment>/<config>/`, and finished configurations are skipped, so the suite is resumable. `make_figures.py` turns those JSON files into every figure and table in this report. The numbers in Section 9 are therefore regenerated from code, not copied by hand.

Legacy preserved. `legacy/` contains the original three files unchanged, so the original program can still be run (`python legacy/main.py`) and compared.

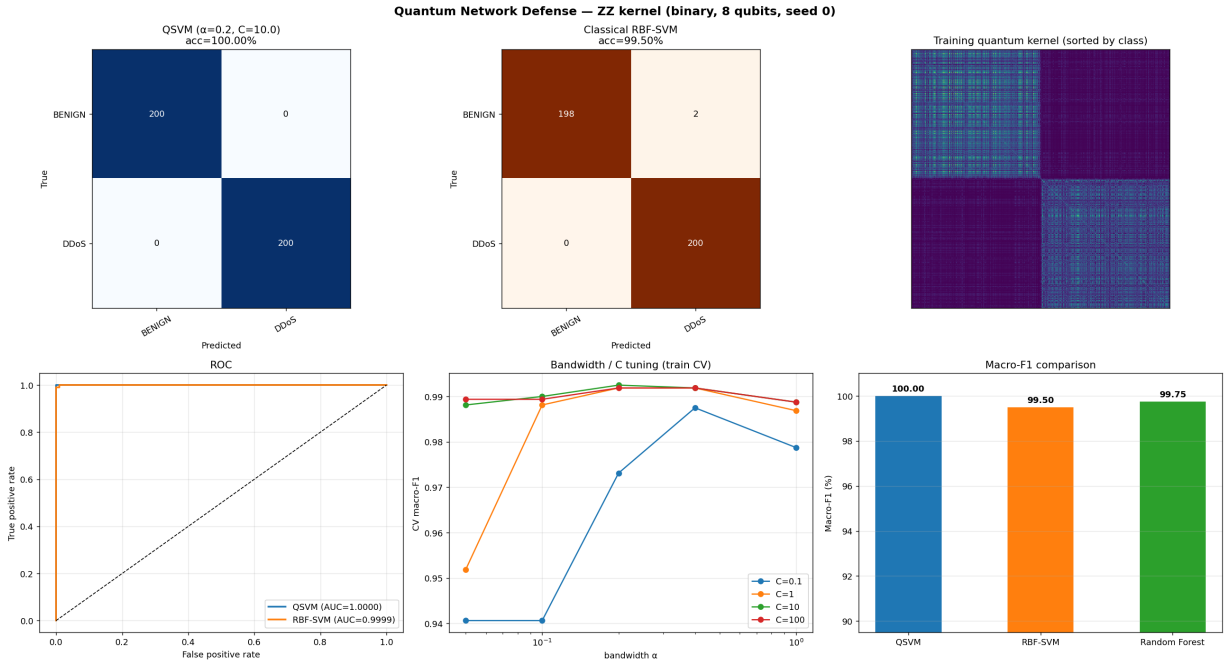


Figure 6: Per-run figure written by `main.py` (binary task, ZZ map with 8 qubits and 2 reps, improved protocol, seed 0). Top: QSVM and RBF confusion matrices, and the training quantum kernel sorted by class (the block structure shows class-aware similarity). Bottom: ROC curves, the CV tuning curves over α for each C, and macro-F1 of all models.

9 Results

Protocol. Every number below is produced by `benchmark.py` and turned into tables and figures by `make_figures.py`. The headline and ablation experiments use 5 seeds (0–4); all other sweeps use 3 seeds (0–2). Each seed redraws the balanced sample, the 80/20 stratified split, the preprocessing, the tuning and the test. With the default 2,000 samples the test set has 400 flows, so one flow is 0.25 accuracy points. "Sig." is the number of seeds in which the exact McNemar test (QSVM vs. RBF-SVM, same test flows) gives $p < 0.05$. Unless stated otherwise, the improved protocol is used: deduplication on the task label, train-only preprocessing with the signed log transform, and 5-fold CV tuning of (α, C) for the QSVM and (C, γ) for the RBF-SVM with the same C grid.

9.1 Headline: the same circuits under the original and the improved protocol

Table 7 runs the paper's three methods (the ZZ entangling map with the paper's circuit settings, the separable R_x map and its 1024-shot version) under both protocols, plus a 16-qubit, 2-repetition ZZ circuit. Figure 7 shows the same data.

Table 7: Test accuracy (%) and QSVM macro-F1 (%), mean ± std over 5 seeds. "Legacy" is the original protocol (`-legacy`); "improved" is the corrected one. ZZ uses the paper's settings: 8 qubits / 2 reps (binary) and 16 qubits / 1 rep (multiclass). R_x uses 16 qubits.

Task	Method	Protocol	QSVM acc.	RBF-SVM acc.	QSVM macro-F1	Sig.
binary	ZZ entangling (paper circuit)	legacy	99.50 ± 0.32	97.35 ± 0.25	99.50 ± 0.32	5/5
binary	ZZ entangling (paper circuit)	improved	99.60 ± 0.41	99.55 ± 0.29	99.60 ± 0.41	0/5
binary	Analytical R_x (separable)	legacy	98.35 ± 0.12	98.60 ± 0.41	98.35 ± 0.12	0/5
binary	Analytical R_x (separable)	improved	99.50 ± 0.32	99.50 ± 0.27	99.50 ± 0.32	0/5
binary	Sampling, 1024 shots (R_x)	legacy	98.30 ± 0.10	98.60 ± 0.41	98.30 ± 0.10	0/5
binary	Sampling, 1024 shots (R_x)	improved	99.30 ± 0.62	99.50 ± 0.27	99.30 ± 0.62	0/5
binary	ZZ, 16 qubits, 2 reps	improved	99.60 ± 0.25	99.50 ± 0.27	99.60 ± 0.25	0/5
multiclass	ZZ entangling (paper circuit)	legacy	95.70 ± 0.56	88.85 ± 1.12	95.68 ± 0.59	5/5
multiclass	ZZ entangling (paper circuit)	improved	90.85 ± 0.85	91.05 ± 0.95	90.85 ± 0.86	0/5
multiclass	Analytical R_x (separable)	legacy	91.70 ± 2.54	88.85 ± 1.12	91.60 ± 2.55	2/5
multiclass	Analytical R_x (separable)	improved	91.05 ± 0.24	91.05 ± 0.95	91.10 ± 0.22	0/5
multiclass	Sampling, 1024 shots (R_x)	legacy	91.35 ± 1.30	88.85 ± 1.12	91.27 ± 1.32	2/5
multiclass	Sampling, 1024 shots (R_x)	improved	88.10 ± 0.41	91.05 ± 0.95	88.16 ± 0.37	2/5
multiclass	ZZ, 16 qubits, 2 reps	improved	91.20 ± 0.84	91.05 ± 0.95	91.22 ± 0.86	0/5

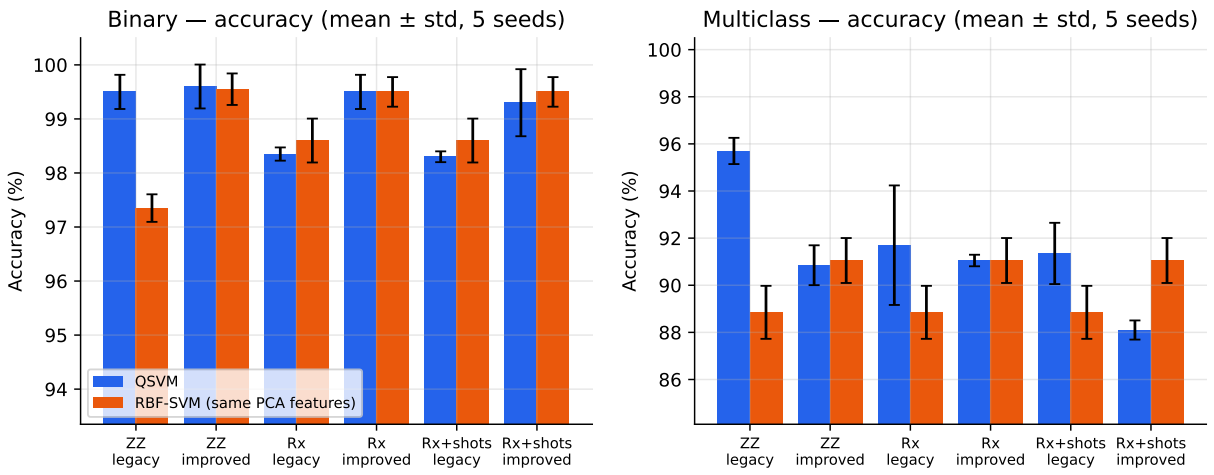


Figure 7: Accuracy of the QSVM (blue) and the RBF-SVM on the same PCA features (orange) under both protocols. Error bars: standard deviation over 5 seeds.

The original protocol reproduces the paper's pattern. Under `-legacy` the ZZ QSVM beats the RBF-SVM by +2.15 points (binary) and +6.85 points (multiclass), significant in 5/5 seeds both times. Summed over the five seeds, the QSVM is right where the RBF-SVM is wrong on 46 binary flows and the reverse happens on 3; in multiclass the counts are 172 and 35. The paper's observation is therefore real *under its protocol*.

The improved protocol removes the gap. With the same circuits, the differences shrink to +0.05 (binary) and −0.20 points (multiclass), and no seed is significant. Pooled over the seeds, the discordant counts are 2 vs. 1 (binary) and 19 vs. 23 (multiclass). The QSVM did not become worse in binary mode (99.50% → 99.60%); the RBF-SVM became much better (97.35% → 99.55%; multiclass 88.85% → 91.05%). The same holds for the 16-qubit, 2-repetition circuit (99.60 vs. 99.50; 91.20 vs. 91.05) and for the separable R_x kernel, which ties the RBF-SVM exactly in both tasks.

Shot noise. In binary mode the 1024-shot R_x kernel is within noise of the exact one. In multiclass mode it costs about 3 points (88.10% vs. 91.05% exact), and the RBF-SVM is significantly better in 2/5 seeds. Section 9.6 explains why this happens at 16 qubits but not at 8.

9.2 Ablation: which fix changes what

Table 8 and Figure 8 add the fixes one at a time to the ZZ pipeline with the paper's circuit settings, starting from the original protocol.

Table 8: Cumulative ablation (5 seeds). "Overlap" is the fraction of test flows with an exact copy in the training set. Gap = QSVM − RBF-SVM accuracy in points.

Task	Step	Overlap	QSVM acc.	RBF-SVM acc.	Gap	Sig.
binary	Original protocol	28.4%	99.50 ± 0.32	97.35 ± 0.25	+2.15	5/5
binary	+ remove duplicates	0.0%	98.85 ± 0.49	96.15 ± 0.51	+2.70	4/5
binary	+ train-only scaling	0.0%	98.65 ± 0.34	96.40 ± 0.89	+2.25	3/5
binary	+ log transform	0.0%	99.15 ± 0.46	99.40 ± 0.68	-0.25	0/5
binary	+ CV tuning (α, C, γ)	0.0%	99.60 ± 0.41	99.55 ± 0.29	+0.05	0/5
multiclass	Original protocol	48.6%	95.70 ± 0.56	88.85 ± 1.12	+6.85	5/5
multiclass	+ remove duplicates	0.8%	89.20 ± 1.02	87.55 ± 1.44	+1.65	2/5
multiclass	+ train-only scaling	0.8%	88.40 ± 1.22	87.35 ± 1.45	+1.05	2/5
multiclass	+ log transform	0.8%	91.40 ± 0.78	90.20 ± 0.58	+1.20	0/5
multiclass	+ CV tuning (α, C, γ)	0.8%	90.85 ± 0.85	91.05 ± 0.95	-0.20	0/5

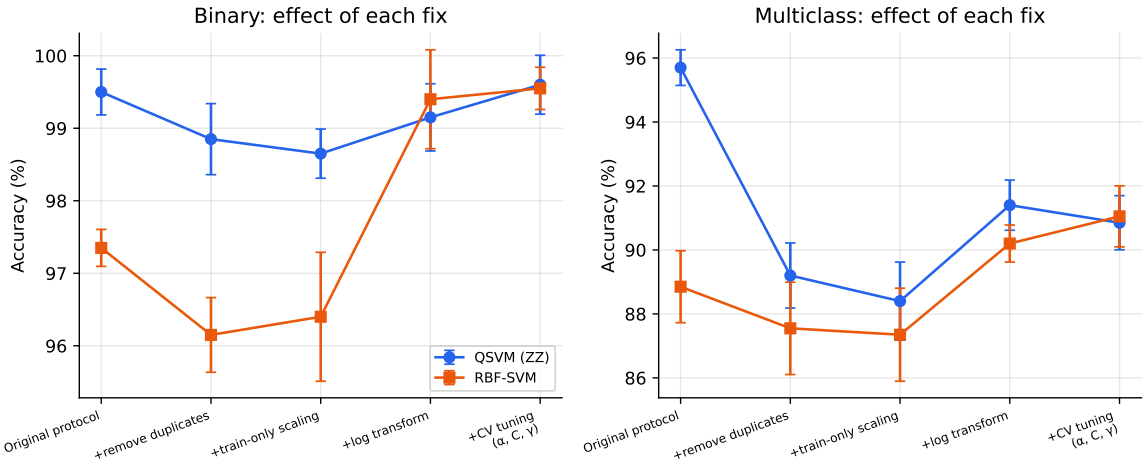


Figure 8: Effect of each fix on QSVM and RBF-SVM accuracy (mean ± std, 5 seeds).

- **Removing duplicates** is the largest single effect in multiclass mode. The QSVM loses 6.5 points (95.70% → 89.20%) and the gap shrinks from +6.85 to +1.65. The QSVM profited more from duplicates than the RBF-SVM: with $\alpha = 1$ its kernel is close to the identity, so a test flow with a training twin gets $K = 1$ to that twin and almost 0 to everything else, which is pure memorisation. In binary mode the overlap was smaller (28%) and both models lose 0.65–1.2 points.
- **Train-only scaling** changes little on its own (≤0.8 points). It is a correctness fix, not a performance one.

- **The log transform** closes the binary gap. The RBF-SVM jumps by 3.0 points (96.40% → 99.40%) and the gap goes from +2.25 to −0.25. Under linear min–max scaling the heavy-tailed features squeeze most flows into a small corner of each PCA component (Figure 2), and the RBF-SVM with a default γ suffers from this much more than the QSVM. Part of the original "advantage" was therefore an encoding artefact that hurt the baseline.
- **CV tuning** gives the last few tenths of a point and removes the remaining multiclass gap (+1.20 → −0.20).

9.3 All metrics for the improved ZZ pipeline

Table 9: All metrics (%), improved protocol, ZZ with the paper's circuit settings, 5 seeds. FAR: benign flows flagged as attacks. DR: attack flows flagged as some attack.

Task	Model	Accuracy	Bal. acc.	Macro-F1	MCC	AUC	FAR	DR
binary	QSVM ZZ (tuned)	99.60 ± 0.41	99.60 ± 0.41	99.60 ± 0.41	99.20 ± 0.81	99.92 ± 0.15	0.30 ± 0.40	99.50 ± 0.63
binary	RBF-SVM (tuned)	99.55 ± 0.29	99.55 ± 0.29	99.55 ± 0.29	99.10 ± 0.58	99.92 ± 0.14	0.40 ± 0.37	99.50 ± 0.63
binary	Random Forest	99.70 ± 0.29	99.70 ± 0.29	99.70 ± 0.29	99.40 ± 0.58	99.99 ± 0.02	0.10 ± 0.20	99.50 ± 0.63
multiclass	QSVM ZZ (tuned)	90.85 ± 0.85	90.86 ± 0.85	90.85 ± 0.86	89.05 ± 1.01	98.71 ± 0.15	1.79 ± 1.46	98.62 ± 0.73
multiclass	RBF-SVM (tuned)	91.05 ± 0.95	91.06 ± 0.95	91.07 ± 0.98	89.29 ± 1.14	98.73 ± 0.28	2.09 ± 2.02	98.98 ± 0.67
multiclass	Random Forest	93.10 ± 0.49	93.11 ± 0.49	93.11 ± 0.48	91.74 ± 0.58	99.03 ± 0.07	2.09 ± 1.52	99.46 ± 0.29

Table 9 confirms the headline with every metric: the QSVM and the RBF-SVM are within one standard deviation of each other on accuracy, balanced accuracy, macro-F1, MCC, AUC, false-alarm rate and detection rate. The Random Forest on all 49 raw features is the best model in both tasks (99.70% binary, 93.10% multiclass). The kernel models only see at most 16 PCA components, and that compression has a cost.

Where the multiclass errors are. Summed over the five seeds, the QSVM's per-class recall is BENIGN 98.2%, UDP 95.5%, Syn 92.5%, LDAP 89.2%, MSSQL 87.4% and Portmap 82.3%. The RBF-SVM's recalls are within 1 point of these for every class. Almost all errors are confusions between the three reflection attacks (LDAP ↔ MSSQL ↔ Portmap), plus Portmap flows predicted as BENIGN or Syn. These are the classes whose flows are 70–94% duplicates in the raw file, so they are the ones that looked easiest under the original protocol. The Random Forest gains most on MSSQL (+4.5), Portmap (+4.5) and UDP (+3.3).

9.4 Bandwidth α

Table 10 and Figure 9 fix α (tuning only C) for the ZZ map with 2 repetitions at 8 and 16 qubits.

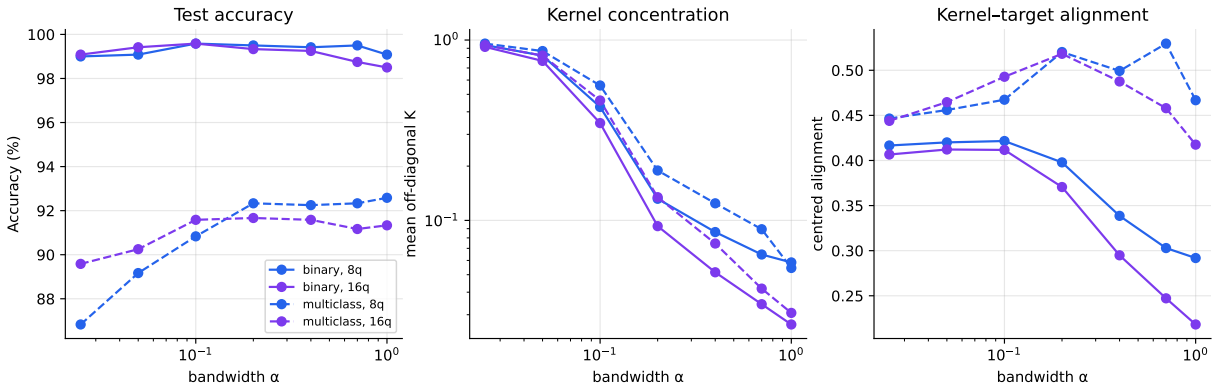


Figure 9: Test accuracy, mean off-diagonal kernel value and centred kernel–target alignment as functions of α (ZZ map, 2 reps, 3 seeds).

Table 10: Bandwidth sweep (3 seeds): QSVM accuracy (%), mean off-diagonal training kernel value, and centred kernel–target alignment.

Task	Qubits	α	QSVM acc.	Mean off-diag. K	Alignment
binary	8	0.025	99.00	9.38e-01	0.417
binary	8	0.05	99.08	8.19e-01	0.420
binary	8	0.1	99.58	4.26e-01	0.422
binary	8	0.2	99.50	1.32e-01	0.398
binary	8	0.4	99.42	8.62e-02	0.339
binary	8	0.7	99.50	6.47e-02	0.303
binary	8	1	99.08	5.85e-02	0.292
binary	16	0.025	99.08	9.17e-01	0.407
binary	16	0.05	99.42	7.66e-01	0.412
binary	16	0.1	99.58	3.46e-01	0.412
binary	16	0.2	99.33	9.29e-02	0.371
binary	16	0.4	99.25	5.16e-02	0.295
binary	16	0.7	98.75	3.43e-02	0.247
binary	16	1	98.50	2.65e-02	0.218
multiclass	8	0.025	86.83	9.56e-01	0.446
multiclass	8	0.05	89.17	8.69e-01	0.456
multiclass	8	0.1	90.83	5.60e-01	0.467
multiclass	8	0.2	92.33	1.89e-01	0.520
multiclass	8	0.4	92.25	1.24e-01	0.499
multiclass	8	0.7	92.33	8.93e-02	0.530
multiclass	8	1	92.58	5.45e-02	0.467
multiclass	16	0.025	89.58	9.37e-01	0.444
multiclass	16	0.05	90.25	8.20e-01	0.465
multiclass	16	0.1	91.58	4.62e-01	0.493
multiclass	16	0.2	91.67	1.34e-01	0.518
multiclass	16	0.4	91.58	7.45e-02	0.488
multiclass	16	0.7	91.17	4.19e-02	0.458
multiclass	16	1	91.33	3.07e-02	0.418

- The mean off-diagonal kernel value falls by more than an order of magnitude from $\alpha = 0.025$ to $\alpha = 1$ (e.g. 0.92 $\rightarrow$ 0.027 at 16 qubits, binary). This is the concentration effect of

Section 2.6, and it is stronger at 16 qubits than at 8.

- At 16 qubits, accuracy peaks at $\alpha \approx 0.1$–0.2 and falls at $\alpha = 1$ (binary 99.58% $\rightarrow$ 98.50%), and alignment falls with it (0.41 $\rightarrow$ 0.22). At 8 qubits the kernel concentrates less, so the curve is flatter. Without a bandwidth, the paper's 16-qubit circuit sits in the worst part of the curve.
- Very small α also hurts, especially in multiclass mode (86.8% at 8 qubits, $\alpha = 0.025$): the kernel approaches the constant 1 and the classes are no longer separated.
- In binary mode, alignment and accuracy peak at the same α (≈ 0.1); in multiclass mode they peak at roughly similar values. The cheap alignment diagnostic (no SVM training needed) is therefore a rough guide for choosing α, but not a substitute for CV.

9.5 Number of qubits

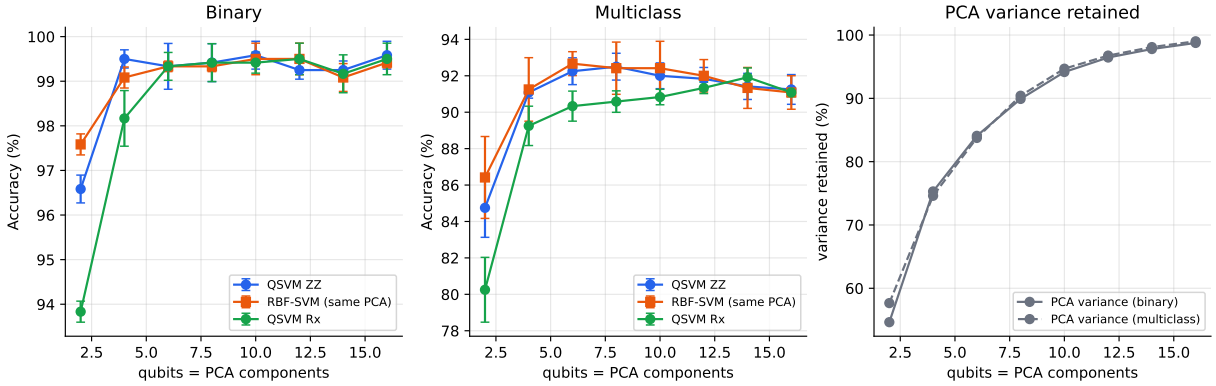


Figure 10: Accuracy vs. number of qubits (= PCA components) for the ZZ and R_x QSVMs and the RBF-SVM on the same components, and the PCA variance retained (3 seeds).

Table 11: Qubit sweep for the ZZ map (2 reps, 3 seeds): PCA variance retained (%), accuracy (%), mean tuned α and geometric difference g against the tuned RBF kernel ($\sqrt{N} = 20$).

Task	Qubits	PCA var.	QSVM ZZ	RBF-SVM	Mean α	g
binary	2	54.6	96.58	97.58	1.000	39.36
binary	4	75.3	99.50	99.08	0.333	10.26
binary	6	84.1	99.33	99.33	0.400	3.75
binary	8	89.9	99.42	99.33	0.333	14.49
binary	10	94.2	99.58	99.50	0.233	10.14
binary	12	96.4	99.25	99.50	0.133	14.06
binary	14	97.8	99.25	99.08	0.133	8.97
binary	16	98.8	99.58	99.42	0.100	2.22
multiclass	2	57.7	84.75	86.42	1.000	31.05
multiclass	4	74.6	91.08	91.25	0.800	32.40
multiclass	6	83.7	92.25	92.67	0.800	9.77
multiclass	8	90.4	92.50	92.42	0.800	17.52
multiclass	10	94.7	92.00	92.42	0.333	3.00
multiclass	12	96.8	91.83	92.00	0.233	7.86
multiclass	14	98.1	91.42	91.33	0.333	2.50
multiclass	16	99.0	91.25	91.08	0.333	3.94

Accuracy saturates early: at 4 qubits the binary task is already at 99.5%, and multiclass accuracy stops improving at 6–8 qubits (84–90% PCA variance). From 4 qubits on, the QSVM and the RBF-SVM are within 0.5 points of each other at every width; only at 2 qubits is the RBF-SVM 1–1.7 points ahead. As the circuit gets wider, the tuned α falls (from 1.0 at 2 qubits to 0.1–0.33 at 12–16 qubits), which is the tuning compensating for concentration. Adding qubits makes the Hilbert space exponentially larger, but that does not show up in accuracy.

9.6 Shot noise

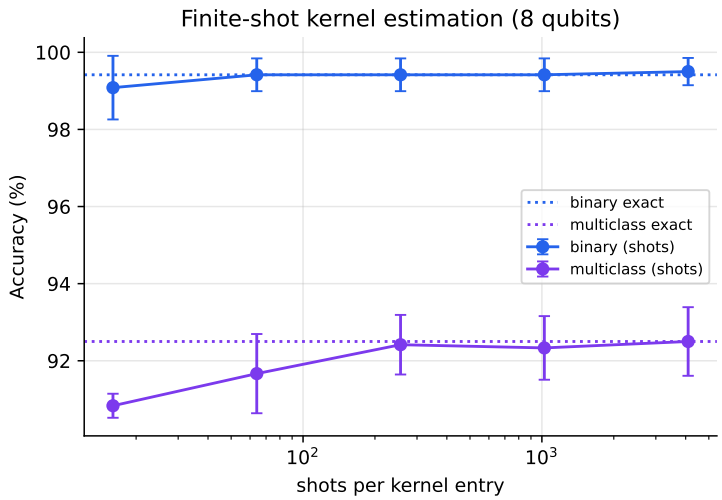


Figure 11: QSVM accuracy vs. shots per kernel entry (ZZ, 8 qubits, 2 reps, 3 seeds). Dotted lines: exact kernel.

Table 12: Shot sweep, ZZ map, 8 qubits (3 seeds): accuracy (%) and mean tuned α.

Task	Shots	QSVM acc.	Mean α
binary	16	99.08 ± 0.82	0.533
binary	64	99.42 ± 0.42	0.400
binary	256	99.42 ± 0.42	0.333
binary	1024	99.42 ± 0.42	0.267
binary	4096	99.50 ± 0.35	0.267
binary	exact	99.42 ± 0.42	0.333
multiclass	16	90.83 ± 0.31	1.000
multiclass	64	91.67 ± 1.03	1.000
multiclass	256	92.42 ± 0.77	1.000
multiclass	1024	92.33 ± 0.82	1.000
multiclass	4096	92.50 ± 0.89	0.800
multiclass	exact	92.50 ± 0.74	0.800

At 8 qubits, 256 shots per entry already match the exact kernel in both tasks, and even 16 shots cost only 0.3 (binary) and 1.7 points (multiclass). The tuned α moves to larger values as the shot count falls (binary: 0.27 at 1024–4096 shots, 0.53 at 16). A likely reason is that larger α pushes most kernel values towards 0, where the binomial variance $K(1-K)/s$ is small, so CV picks a bandwidth that limits the noise.

The 16-qubit R_x result in Table 7 is different: there, 1024 shots cost 3 points in multiclass mode, while in binary mode they cost nothing. With 1024 shots each entry has a standard

deviation of up to $\sqrt{0.25/1024} \approx 0.016$. The binary classes are far apart in kernel space, but the three reflection attacks are separated by small kernel differences that are apparently of the same order. We have not isolated why the 8-qubit ZZ kernel is robust and the 16-qubit R_x kernel is not. The practical lesson is that the shot budget needed depends on the circuit and the task, and has to be measured before trusting a hardware result.

Projecting the noisy kernel onto the PSD cone does not help. An estimated Gram matrix is not positive semi-definite. On 1,600 multiclass training flows (16 qubits, R_x, 1024 shots), the negative eigenvalues sum to between -83 and -319 depending on α, so the SVM dual is no longer convex. The standard remedy is to clip the negative eigenvalues of the training kernel to zero [16], available as `-psd`. On the multiclass headline it made results worse in 5/5 seeds (88.10% → 84.65%). Clipping changes only the training kernel, while the test kernel keeps its noise, so the two no longer describe the same geometry. The option is therefore off by default.

9.7 Circuit architecture and feature set

Table 13: ZZ QSVM accuracy (%) by number of repetitions and entanglement topology (8 qubits, 3 seeds).

Task	Reps	Linear	Circular	Full
binary	1	99.25 ± 0.20	99.25 ± 0.20	99.25 ± 0.41
binary	2	99.42 ± 0.42	99.50 ± 0.35	99.33 ± 0.24
binary	3	99.42 ± 0.42	99.17 ± 0.24	99.42 ± 0.12
multiclass	1	92.58 ± 0.66	92.50 ± 0.71	92.50 ± 0.54
multiclass	2	92.50 ± 0.74	92.58 ± 0.82	92.00 ± 0.35
multiclass	3	92.42 ± 1.45	92.75 ± 0.89	91.92 ± 0.12

No architecture stands out (Table 13). All nine combinations of 1–3 repetitions and linear, circular or full entanglement are within 0.35 points of each other in binary mode and 0.85 points in multiclass mode, which is within one standard deviation. More entanglement or more repetitions, which make the kernel harder to simulate classically, do not make it more accurate here.

Table 14: Feature set (8 qubits, 3 seeds): accuracy (%).

Task	Feature set	QSVM ZZ	RBF-SVM	Random Forest
binary	paper (61 listed, 49 non-constant)	99.42 ± 0.42	99.33 ± 0.12	99.50 ± 0.20
binary	all numeric CICFlowMeter columns	99.42 ± 0.31	99.50 ± 0.54	99.67 ± 0.12
multiclass	paper (61 listed, 49 non-constant)	92.50 ± 0.74	92.42 ± 1.43	93.17 ± 0.59
multiclass	all numeric CICFlowMeter columns	93.92 ± 0.72	94.25 ± 0.35	94.67 ± 0.24

Using all 77 numeric CICFlowMeter columns instead of the paper's list (Table 14) adds 1.4–1.8 points in multiclass mode for *every* model. The extra Subflow, `Init_Win_bytes` and Active/Idle columns carry real information about the reflection attacks. The gap between the models does not change.

9.8 Learning curves

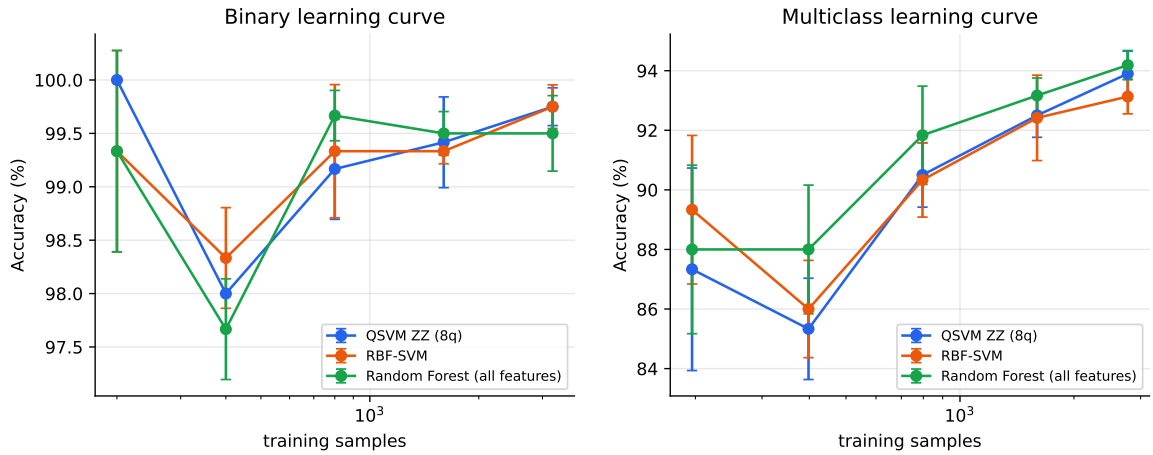


Figure 12: Accuracy vs. number of training flows (8 qubits, 3 seeds). In multiclass mode the largest setting is capped at 2,774 training flows by the 578 unique LDAP flows.

The QSVM and RBF-SVM curves overlap at every size from 200 to 3,200 training flows (Figure 12). Neither model learns faster from few examples, which is sometimes proposed as an advantage of quantum kernels. Multiclass accuracy is still rising at the largest size (93.9% QSVM, 93.1% RBF-SVM, 94.2% Random Forest), so more data would help all models, but deduplication leaves only 578 LDAP flows in this file.

9.9 Kernel diagnostics

Table 15: Kernel diagnostics on the training kernel (headline runs, mean over 5 seeds): off-diagonal mean and standard deviation, centred alignment of the quantum and the tuned RBF kernel, and the geometric difference $g(K_{\mathrm{RBF}} \| K_Q)$ on 400 training flows.

Task	Kernel	Off-diag. mean	Off-diag. std	Align. K_Q	Align. K_{RBF}	$g(K_{\mathrm{RBF}} \| K_Q)$
binary	ZZ, original (α=1)	0.0781	0.239	0.297	0.233	65.74 (vs $\sqrt{N}$=20)
binary	ZZ, tuned α	0.0971	0.249	0.353	0.348	10.85 (vs $\sqrt{N}$=20)
binary	ZZ 16q/2 reps, tuned α	0.427	0.242	0.433	0.296	4.03 (vs $\sqrt{N}$=20)
binary	R_x, tuned α	0.294	0.267	0.370	0.296	2.90 (vs $\sqrt{N}$=20)
multiclass	ZZ, original (α=1)	0.0701	0.23	0.494	0.479	111.97 (vs $\sqrt{N}$=20)
multiclass	ZZ, tuned α	0.0867	0.2	0.467	0.484	13.76 (vs $\sqrt{N}$=20)
multiclass	ZZ 16q/2 reps, tuned α	0.084	0.218	0.487	0.484	9.19 (vs $\sqrt{N}$=20)
multiclass	R_x, tuned α	0.379	0.287	0.516	0.484	3.08 (vs $\sqrt{N}$=20)

Table 15 explains the results.

- **Alignment.** With tuned α, the quantum kernels are aligned with the labels about as well as the tuned RBF kernel is (binary 0.35–0.43 vs. 0.30–0.35; multiclass 0.47–0.52 vs. 0.48). Equal alignment predicts equal accuracy, which is what we observe.
- **Geometric difference.** For every tuned quantum kernel, g is well below $\sqrt{N} = 20$: 2.9–3.1 for R_x, 4.0–9.2 for the 16-qubit ZZ map and 10.9–13.8 for the paper's ZZ settings. By Huang et al. [15], a classical kernel trained on the same data can then match the quantum kernel's predictions up to a small factor, so no large advantage is possible on this data, whatever the labels.

- **The original protocol** gives very large g (65.7 binary, 112.0 multiclass). This does *not* indicate quantum power: under the original encoding the reference RBF kernel is itself badly scaled, and with $\alpha = 1$ the quantum kernel is close to the identity, which makes *any* smooth kernel look geometrically "different". A large g is necessary for an advantage, not sufficient, and here the labels do not need the extra geometry.

Summary of the results

Under a leakage-free protocol with equal tuning, the ZZ quantum-kernel SVM and the classical RBF-SVM are statistically indistinguishable on every metric, every circuit architecture, every width from 4 to 16 qubits and every training-set size tested. The original advantages (+2.15 and +6.85 points) come from duplicate leakage, which mostly helped the QSVM, and from a linear encoding of heavy-tailed features, which mostly hurt the RBF baseline. A Random Forest on all raw features is the best model in both tasks.

10 Discussion

10.1 Does the quantum kernel help on this task?

No, not on this data. Under the corrected protocol the ZZ quantum kernel matches a tuned classical RBF kernel on the same PCA features. It does not beat it on any metric, circuit architecture, qubit count or training-set size we tested, and a Random Forest on all raw features beats both. This agrees with large systematic benchmarks of quantum models on classical data [17], which found that well-tuned classical baselines usually match or beat them.

This is a negative result about *this* dataset and feature map, not about quantum kernels in general. The diagnostics show why it comes out this way:

1. **The problem is easy and low-dimensional.** Binary accuracy is 99.5% with only 4 PCA components, and every model saturates at 6–8 components. There is little left for a more expressive kernel to find.

2. **The quantum kernel's geometry is classically reachable.** With tuned α, the geometric difference to the RBF kernel is 3–14, well below $\sqrt{N} = 20$. By Huang et al. [15] a classical kernel can then reproduce the quantum kernel's predictions, so no large advantage is possible, whatever the labels.

3. **The useful regime of the ZZ map is the classical-looking one.** The tuning selects small α for the 16-qubit, 2-repetition circuit (0.05–0.4). There, the phases are small and the kernel behaves like a smooth, RBF-like similarity. The large-α regime, where the circuit is conjectured to be hard to simulate, is exactly where it concentrates and loses accuracy (Figure 9).

4. **Entanglement is not used.** The separable R_x kernel, which is a classical closed-form expression, performs the same as the entangling ZZ kernel, and more repetitions or denser entanglement do not change accuracy (Table 13).

5. **Flow statistics have no quantum structure.** Known quantum advantages for kernels use data whose labels are generated by a quantum process or encode a problem that is hard classically (e.g. discrete logarithms). CICFlowMeter statistics are classical summaries of packet counts and timings, and the ZZ feature map was not designed around them.

10.2 What went wrong in the original evaluation

The original advantages were produced by the evaluation, not by the model. Ranked by effect:

1. **Duplicate leakage** (up to 49% of test flows with a training twin). It inflated the QSVM more than the RBF-SVM: a nearly diagonal kernel memorises twins perfectly.
2. **Linear encoding of heavy-tailed features.** It hurt the RBF baseline much more than the QSVM (3 points in binary mode), creating most of the binary gap.
3. **Unequal tuning.** The RBF-SVM was run with default C and γ. Comparing a configured model against a default one measures the configuration, not the model.
4. **Single small test sets.** The binary ZZ result rested on 165 test flows, where "98.79% vs. 96.97%" is 163 vs. 160 correct. No repetition or significance test was reported.

These are general lessons for quantum-ML benchmarks on network data: deduplicate before splitting, fit every transform on training data only, give the classical baseline the same tuning budget, report several seeds with a paired test, and include a strong non-kernel baseline.

10.3 Threats to validity

- **Simulation only.** All kernels are exact or binomially sampled statevector simulations. Hardware adds gate errors, decoherence and readout noise. The shot experiments model only statistical noise, and even that costs 3 points in the 16-qubit multiclass setting (Section 9.6).
- **One capture, contiguous slices.** Each class in `output.csv` is a contiguous slice of one capture day, in some cases spanning about a second. Random train/test splits within such a slice are easier than training on one day and testing on another. Cross-day or cross-dataset evaluation would be the stronger test, and it would make all models' accuracy lower.
- **Balanced sampling.** Real traffic is heavily imbalanced, so the false-alarm rate would matter much more than accuracy. The FAR is reported, but not at a realistic base rate.
- **Statistical power.** Each seed tests 400 flows, so per-seed McNemar tests only detect differences of roughly 2 points or more. Pooled over 5 seeds, the discordant counts are nearly balanced (2 vs. 1 binary, 19 vs. 23 multiclass), so a large hidden advantage is very unlikely, but differences below about 1 point cannot be ruled out.
- **Shared preprocessing inside CV.** PCA and scaling are fitted once on the full training split, and the CV folds for tuning reuse that fit. The validation folds therefore contribute to the unsupervised preprocessing. This affects the QSVM and the RBF-SVM equally and does not touch the test set.
- **Bandwidth grid boundary for R_x.** The tuned α of the R_x kernel was 1.0, the top of the default grid, in all 10 headline runs. Rerunning the 16-qubit R_x headline with $\alpha \in \{\ldots, 1.5, 2, 3\}$ selects $\alpha = 1.5$–3 and raises accuracy slightly (binary 99.50% $\rightarrow$ 99.60%, multiclass 91.05% $\rightarrow$ 91.65%), still with no significant seed against the RBF-SVM ($p \geq 0.125$). The conclusion is unchanged, but the reported R_x numbers are slightly conservative. For the ZZ map the tuned α lies inside the grid.
- **Ambiguous rows.** After deduplication, 177 multiclass rows (2 binary) share their feature vector with a row of a different class. They are kept as genuine ambiguity and cause the 0.8% residual train/test overlap in multiclass mode.

10.4 Future work

1. **Harder data:** the 12 January capture day of CIC-DDoS2019 (DNS, NTP, SSDP, SNMP and other near-identical reflection attacks), cross-day train/test splits, and other datasets such as CICIDS2017 or UNSW-NB15. `-csv` already accepts any CICFlowMeter file.
2. **Trainable feature maps** that maximise kernel–target alignment (`evaluation.centered_alignment` is already implemented).
3. **Projected quantum kernels** [15], which are less prone to concentration.
4. **Noise models and mitigation:** depolarising and readout noise on the Cirq backend, and noise-aware kernel post-processing that treats the training and test kernels consistently. Clipping only the training kernel's spectrum made results worse here.
5. **Hardware runs** of a subset of kernel entries using `build_kernel_circuit()`.
6. **Imbalanced evaluation:** train on balanced data, test at the natural class ratio, and report precision at a fixed false-alarm rate.

10.5 Conclusion

The project now has a correct, fast and reproducible quantum-kernel pipeline. Kernels are verified against Cirq, 16-qubit kernels take seconds instead of days, preprocessing is leakage-free, tuning is equal, and the evaluation uses several seeds with significance tests. With that pipeline, the reported quantum advantage on CIC-DDoS2019 does not survive. The ZZ quantum kernel SVM is as good as, but not better than, a tuned classical RBF-SVM, and both are slightly behind a Random Forest on all features. The tools built here (kernel diagnostics, the geometric-difference test, the benchmark suite) can be used to test whether a quantum kernel helps on harder data, and to find out quickly when it does not.

A Command-line reference

Option	Default	Meaning
`-csv PATH`	`./output.csv`	input file (original)
`-samples N`	2000	total balanced sample size (original)
`-mode`	`binary`	`binary` or `multiclass` (original)
`-n-qubits n`	16	qubits = PCA components (original)
`-n-reps r`	2	feature-map repetitions (original)
`-sampling`	off	finite-shot kernel (original flag; now fast binomial emulation unless `-backend cirq`)
`-shots s`	1024	shots per kernel entry
`-kernel`	`zz`	`zz` (entangling) or `rx` (separable, paper Method 1)
`-entanglement`	`linear`	`linear`, `circular`, `full`
`-backend`	`fast`	`fast` (NumPy, exact) or `cirq`
`-alphas a...`	0.05 0.1 0.2 0.4 1	bandwidth grid for CV
`-Cs c...`	0.1 1 10 100	C grid for CV (QSVM and RBF)
`-no-tune`	off	use `-alpha`/`-C` and an untuned RBF (`C=1, gamma='scale'`)
`-alpha a, -C c`	1, 1	fixed values when `-no-tune`
`-seeds k...`	42	one full run per seed; mean ± std reported
`-features`	`paper`	`paper` (61 listed columns) or `all` (77)
`-keep-duplicates`	off	do not deduplicate
`-no-log`	off	disable the signed log transform
`-legacy-scaling`	off	original preprocessing (global MinMax before split)
`-legacy`	off	= `-legacy-scaling -keep-duplicates -no-tune -alpha 1 -C 1`
`-extra-baselines`	off	also train a Random Forest on all raw features
`-out-dir DIR`	`outputs/<tag>`	where `results.json` and figures go
`-quiet`	off	suppress per-class reports and tuning logs

B File-by-file reference

`data_loader.py` `load_csv(csv_path, mode, max_samples, balance, dedup, feature_set, seed, verbose)` → `(X_raw, y, label_names, info)`. Also `duplicate_overlap(X_train, X_test)`. Constants `FEATURE_COLS` (61 names), `META_COLS`.

`preprocessing.py` `QuantumPreprocessor(n_components, log_transform, clip, seed)` with `fit`, `transform` (angles), `transform_pca` (scores for classical models), `explained_variance`. `LegacyPreprocessor` and `legacy_global_scale` reproduce the original behaviour.

`qsvm.py` `entangling_pairs`; Cirq maps `ZZFeatureMap`, `RxFeatureMap` (`build_encoding_circuit`, `build_kernel_circuit`); `FastZZStatevector` (`phases`, `states`); `QuantumKernel` (`states`, `compute_kernel_value`, `compute_kernel_matrix`); `QSVM` (`fit`, `predict`, `predict_proba`, `decision_function`, `score`, `test_kernel`).

`evaluation.py` `classification_metrics`, `mcnemar`, `cv_score_precomputed`, `tune_quantum`, `tune_rbf`, `offdiag_mean/std`, `centered_alignment`, `geometric_difference`, `rbf_kernel_matrix`, `kernel_diagnostics`, `standardize`.

`main.py` `run_once(args, seed)`, `plot_run`, `summarize`, `main(argv)`.

`benchmark.py` the nine experiments; `-only`, `-quick`.

`make_figures.py` builds `report/figures` and `report/tables`.

`tests/test_core.py` 17 unit tests.

`legacy/` the original `data_loader.py`, `qsvm.py`, `main.py`, and the original result figure.

`TESTING.md` exact commands to test everything.

C Reproduction commands

```
cd ~/Desktop/qcnn && source venv/bin/activate
python -m unittest discover -s tests -v                # 19 tests, ~15 s
python main.py --legacy --n-qubits 8                   # original protocol
python main.py --n-qubits 8                            # improved, binary
python main.py --mode multiclass --n-qubits 16 --n-reps 1
python main.py --n-qubits 8 --seeds 0 1 2 3 4 --extra-baselines
python main.py --kernel rx --n-qubits 16               # paper Method 1
python main.py --kernel rx --n-qubits 16 --sampling    # paper Method 2
python benchmark.py && python make_figures.py          # all report results
cd report && pdflatex report.tex && pdflatex report.tex # this PDF
```

`TESTING.md` lists more commands (ablation, Cirq cross-check, real circuit sampling) with expected runtimes.

D Glossary

Alignment Cosine similarity between a centred kernel matrix and the centred label kernel.

Bandwidth (α, γ) The scale at which a kernel considers points similar.

CICFlowMeter Tool that turns packet captures into bidirectional flow statistics.

Concentration Off-diagonal kernel values collapsing towards a constant (here, 0).

DR / FAR Detection rate / false-alarm rate.

Feature map Circuit $U(\boldsymbol{x})$ that encodes data into a quantum state.

Fidelity kernel $K(\boldsymbol{a}, \boldsymbol{b}) = |\langle\phi(\boldsymbol{a})|\phi(\boldsymbol{b})\rangle|^2$.

Geometric difference Huang et al.'s test of whether a classical kernel can match a quantum one on given data.

Leakage Information from the test set influencing training.

McNemar test Paired test of whether two classifiers err on different samples.

PCA Principal component analysis: projection onto the directions of largest variance.

Statevector The 2^n complex amplitudes describing an n-qubit pure state.

ZZ feature map Havlíček et al.'s entangling encoding with Z_iZ_j interaction terms.

References

[1] A. Topkar, A. Abhyankar, A. Phatak, N. Agte, M. Nivelkar, S. Bhirud. Network Intrusion Detection Using Quantum Machine Learning: A Comparative Study of Analytical Kernel, Sampling, and ZZ Entangling Feature Map Methods. Manuscript (`qc_comsis_2.pdf`).

[2] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, J. M. Gambetta. Supervised learning with quantum-enhanced feature spaces. *Nature* 567, 209–212 (2019).

[3] I. Sharafaldin, A. H. Lashkari, S. Hakak, A. A. Ghorbani. Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy. *Proc. IEEE ICCST*, 1–8 (2019).

[4] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. *Proc. ICISSP*, 108–116 (2018).

[5] C. Cortes, V. Vapnik. Support-vector networks. *Machine Learning* 20(3), 273–297 (1995).

[6] J. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. *Advances in Large Margin Classifiers*, 61–74 (1999).

[7] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *JMLR* 12, 2825–2830 (2011).

[8] Cirq Developers. Cirq: A Python framework for creating, editing, and invoking quantum circuits. https://quantumai.google/cirq.

[9] M. A. Nielsen, I. L. Chuang. *Quantum Computation and Quantum Information.* Cambridge University Press (2010).

[10] M. Schuld, N. Killoran. Quantum machine learning in feature Hilbert spaces. *Phys. Rev. Lett.* 122, 040504 (2019).

[11] M. Schuld. Supervised quantum machine learning models are kernel methods. arXiv:2101.11020 (2021).

[12] S. Thanasilp, S. Wang, M. Cerezo, Z. Holmes. Exponential concentration in quantum kernel methods. *Nature Communications* 15, 5200 (2024).

[13] R. Shaydulin, S. M. Wild. Importance of kernel bandwidth in quantum machine learning. *Phys. Rev. A* 106, 042407 (2022).

[14] A. Canatar, E. Peters, C. Pehlevan, S. M. Wild, R. Shaydulin. Bandwidth enables generalization in quantum kernel models. *Transactions on Machine Learning Research* (2023).

[15] H.-Y. Huang, M. Broughton, M. Mohseni, R. Babbush, S. Boixo, H. Neven, J. R. McClean. Power of data in quantum machine learning. *Nature Communications* 12, 2631 (2021).

[16] T. Hubregtsen, D. Wierichs, E. Gil-Fuster, P.-J. H. S. Derks, P. K. Faehrmann, J. J. Meyer. Training quantum embedding kernels on near-term quantum computers. *Physical Review A* 106, 042431 (2022).

[17] J. Bowles, S. Ahmed, M. Schuld. Better than classical? The subtle art of benchmarking quantum machine learning models. arXiv:2403.07059 (2024).

[18] C. Cortes, M. Mohri, A. Rostamizadeh. Algorithms for learning kernels based on centered alignment. *JMLR* 13, 795–828 (2012).

[19] Q. McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12(2), 153–157 (1947).

[20] T. G. Dietterich. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation* 10(7), 1895–1923 (1998).

[21] S. Kaufman, S. Rosset, C. Perlich, O. Stitelman. Leakage in data mining: formulation, detection, and avoidance. *ACM TKDD* 6(4), 15 (2012).

[22] G. Engelen, V. Rimmer, W. Joosen. Troubleshooting an intrusion detection dataset: the CICIDS2017 case study. *IEEE Security and Privacy Workshops*, 7–12 (2021).

[23] L. Breiman. Random forests. *Machine Learning* 45, 5–32 (2001).

[24] J. Preskill. Quantum computing in the NISQ era and beyond. *Quantum* 2, 79 (2018).

[25] R. Sommer, V. Paxson. Outside the closed world: On using machine learning for network intrusion detection. *IEEE S&P*, 305–316 (2010).